IPre Sueño-Vigilia y Aprendizaje Composicional · IMT2985

¿Qué gana y qué pierde un modelo al discretizar?

Discretización de espacios continuos: evidencia, mecanismos y límites

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl

Octubre de 2026

Contenidos

Cómo leer este capítulo **3**
Las preguntas que responde 3
Qué se da por sabido 3
Convenciones 4
Calendario sugerido (una semana) 4
I Introducción: el paquete de la discretización **5**
I.1 La pregunta, y por qué está mal planteada 5
I.2 El paquete 5
I.3 Cómo usamos la evidencia 7
II Fundamentos **9**
II.1 Espacio latente 9
II.2 Información: cuánto cabe en un código 10
II.3 Regresión frente a clasificación 10
II.4 Aprendizaje por refuerzo y modelos del mundo 11
II.5 Aprender sin etiquetas 12
II.6 Cómo leer un resultado experimental 13
III Cómo se discretiza un espacio continuo **14**
III.1 Cuatro maneras de discretizar un vector 14
III.2 Otros niveles donde se puede discretizar 16
III.3 One-hot o vector: la misma información, distinta geometría 17
III.4 El precio: cómo pasar el gradiente 18
IV Evidencia: dónde ayuda y dónde no **20**
IV.1 La línea de Dreamer: de gaussiano a categórico 20
IV.2 El experimento más limpio: Meyer et al. 22
IV.3 Control continuo: DCWM 25
IV.4 Ramificaciones: elegir en vez de promediar 26
IV.5 Robustez 28
IV.6 Dónde gana lo continuo 28
IV.7 Resumen del capítulo 30
V Seguir el error **31**
V.1 El problema: en continuo, el error es invisible 31
V.2 Detectar y corregir: la geometría de los márgenes 32
V.3 Corregir en cadena: errores que no se acumulan 33
V.4 Localizar y contar: el error como evento 36

V.5 El precio y los límites 37
V.6 ¿Es exclusivo de lo discreto? 37
VI ¿Es la discretización? 39
VI.1 Desarmar el paquete 39
VI.2 Un experimento para desarmar el paquete (P3) 40
VI.3 ¿Hay un principio más allá de la discreción? (A1) 42
VII Colapso y error acumulado 46
VII.1 Una familia de colapsos 46
VII.2 ¿Hace más difícil el colapso posterior lo discreto? (P1) 48
VII.3 Colapso del diccionario (P5) 49
VII.4 ¿Errores acumulados = colapso? (N) 50
VIII JEPA y discretización 53
VIII.1 Recordatorio: qué hace un JEPA 53
VIII.2 Quién discretiza qué, y por qué 54
VIII.3 Por qué discretizan, en resumen 55
IX ¿Cuándo conviene discretizar? 57
IX.1 Criterios a favor 57
IX.2 Criterios en contra 58
IX.3 Un árbol de decisión 58
IX.4 Un mapa 58
IX.5 Límites del marco 58
X Conclusiones 61
X.1 Respuestas cortas 61
X.2 Qué significa para nuestro proyecto 62
X.3 Preguntas para la reunión 63
Glosario 64
Referencias 65

Cómo leer este capítulo

Este texto reúne, en forma de capítulo de libro, lo que se sabe hoy sobre una pregunta que nos acompaña desde hace varias semanas: **¿por qué, al discretizar ciertos espacios, mejoran los algoritmos?** No intenta resumir todos los papers del área. Intenta que, al terminar, se pueda responder con argumentos a las preguntas que nos dejó el profesor y a dos preguntas que quedaron abiertas.

Las preguntas que responde

Id	Pregunta	Capítulo
P1	Al entrenar por gradiente aparece el colapso (en especial, el colapso posterior). ¿Es más difícil que ocurra con una representación discreta? ¿Ayuda partir con códigos bien separados?	VII
P2	¿Qué evidencia hizo pensar que discretizar ayuda, sobre todo en aprendizaje por refuerzo? ¿Dónde gana lo continuo?	IV
P3	Si la discretización resuelve un problema, ¿qué experimento lo comprueba o lo desmiente? ¿Qué produce la mejora?	VI
P4	En la familia JEPA, ¿alguien discretiza? ¿Por qué?	VIII
P5	¿En qué modelos ocurre el colapso del diccionario (*codebook collapse*)?	VII
N	Nota del profesor: *«accumulated errors =?= collapse»*.	V y VII
A1	¿Es la discreción misma la que da ventajas, o alguna propiedad que viene con ella? ¿Hay un principio más allá?	VI
A2	¿Qué comparten los espacios donde conviene discretizar, y aquellos donde no conviene?	IX
E	¿Qué significa que la discretización «permite seguir el error»?	V

Las preguntas A1 y A2 no tienen una respuesta establecida en la literatura. Para ellas, el capítulo llega a una conjetura respaldada por la evidencia y propone cómo ponerla a prueba.

Qué se da por sabido

Se da por conocido lo que está en nuestros apuntes: el VAE, el VQ-VAE, la Gumbel-Softmax, el colapso posterior, DreamerV3, LPN, NLI e I-JEPA. Cada uno se recuerda en pocas líneas cuando aparece. Todo lo demás se explica antes de usarse, en recuadros como este:

> **Idea base** *Ejemplo*
>
> Una *idea base* explica en pocas líneas un concepto que el texto necesita y que no vimos en el curso. Si ya se conoce, se puede saltar.

Convenciones

- **Color.** En todas las figuras, **azul** es continuo y **naranjo** es discreto.
- **Certeza.** Las afirmaciones importantes llevan una etiqueta. ESTABLECIDO significa que hay un experimento controlado que la respalda. SUGERIDO, que hay evidencia indirecta. CONJETURA, que es una propuesta nuestra.
- **Piezas.** La introducción define seis «piezas» de la discretización, numeradas del 1 al 6. Cuando un resultado se atribuye a una pieza, se marca con su número.
- **Recuadros.** Cada capítulo abre con **En una frase** y cierra con **Qué nos llevamos**. Leídos seguidos, los cierres forman un resumen. Los recuadros **Para pensar** son preguntas cortas para detenerse un momento; responderlas no es necesario para seguir.

Calendario sugerido (una semana)

Día 1	Introducción y capítulo II (Fundamentos).
Día 2	Capítulos III (Cómo se discretiza) y IV, hasta Meyer et al.
Día 3	Resto del capítulo IV y capítulo V (Seguir el error).
Día 4	Capítulos VI y VII.
Día 5	Capítulos VIII, IX y X.

Capítulo I

Introducción: el paquete de la discretización

En una frase *Discretizar no es un solo cambio, sino varios a la vez; para entender por qué ayuda (o perjudica) hay que separar esos cambios y preguntarse cuál de ellos necesita cada problema.*

I.1 La pregunta, y por qué está mal planteada

En el curso vimos un caso que parecía resolver la cuestión. LPN (Macfarlane & Bonnet, 2025) representa un programa como un punto en un espacio continuo y lo busca por gradiente. NLI (Macfarlane et al., 2026), de casi los mismos autores, aprende un vocabulario discreto de operaciones, las encadena en secuencias y generaliza mucho mejor a combinaciones nuevas. La conclusión tentadora es «lo discreto generaliza mejor».

Pero NLI no cambia una sola cosa respecto de LPN. Sus símbolos son discretos, **y además** sus programas tienen largo variable, **y además** el decodificador es un intérprete que lee símbolo por símbolo. Cualquiera de esos cambios podría explicar la mejora. Por eso, en el reporte de la semana 9 quedó anotada una duda: ¿es necesariamente el discretizar lo que mejora los algoritmos, o es algún principio subyacente que la discretización pone en práctica?

Este capítulo toma esa duda en serio. La tesis es que la pregunta «¿discreto o continuo?» está mal planteada, porque «discretizar» no nombra una intervención, sino un paquete de intervenciones que casi siempre viajan juntas.

I.2 El paquete

Pensemos en lo más simple: un codificador produce un vector continuo $z \in \mathbb{R}^d$, y un cuantizador q lo reemplaza por uno de K códigos posibles (Figura 1). ¿Qué cambió? Al menos seis cosas:

1. ❶ **Capacidad.** Un código de K entradas lleva como máximo $\log_2 K$ bits. Un vector continuo, en principio, puede llevar muchos más. Discretizar impone un cuello de botella de información.
2. ❷ **Geometría.** El código se puede entregar a la red siguiente como un vector *one-hot* (todos los códigos a la misma distancia) o como el vector del diccionario que le corresponde (códigos cercanos y lejanos). No es un detalle: veremos que cambia los resultados.

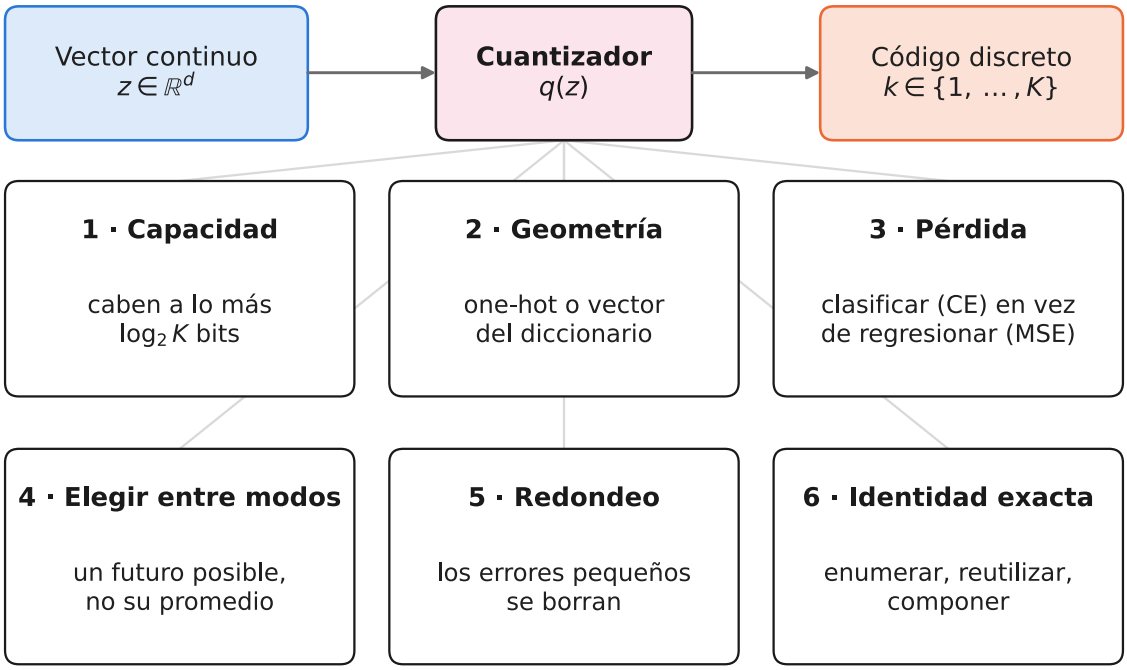


Figura 1. Las seis piezas que cambian a la vez al discretizar. A lo largo del capítulo, cada resultado de la literatura se atribuye a una (o a varias) de estas piezas.

3. ❸ **Pérdida.** Si lo que se predice es un código, la pérdida natural es la entropía cruzada (clasificar) y no el error cuadrático (regresionar). Son problemas de optimización distintos.
4. ❹ **Elegir entre modos.** Un predictor que entrega una distribución sobre códigos puede repartir probabilidad entre futuros distintos. Uno que entrega un vector tiende a entregar su promedio.
5. ❺ **Redondeo.** Al proyectar al código más cercano, un error pequeño desaparece. Esto es lo que permite *seguir* el error, la idea a la que dedicamos el capítulo V.
6. ❻ **Identidad exacta.** Dos estados son el mismo código o no lo son. Eso permite enumerar, reutilizar, comparar y componer, cosas que en un espacio continuo requieren elegir una tolerancia arbitraria.

Cuando un paper reporta que «discretizar mejoró el resultado», casi nunca dice cuál de las seis piezas fue la responsable, porque cambió las seis a la vez. Separarlas es justamente lo que pide la pregunta P3, y es la condición para responder la pregunta A1: si cada beneficio se puede atribuir a una pieza, y cada pieza se puede conseguir por otra vía, entonces el principio no es «discretizar», sino lo que la pieza hace.

> **Para pensar** *Antes de seguir, ¿cuál de las seis piezas crees que explica mejor la diferencia entre LPN y NLI? Vale la pena anotar la respuesta y revisarla al final del capítulo VI.*

I.3 Cómo usamos la evidencia

El profesor compartió un documento generado con IA que reúne pros y contras de las representaciones discretas en modelos del mundo (Petrache, 2026). Es útil como catálogo, pero no tiene hilo: enumera resultados de papers distintos sin decir qué mecanismo explica cada uno. Aquí lo usamos como materia prima. Cada pro y cada contra se lee como evidencia sobre una pieza del paquete, y cada paper como un experimento que aísla, o no, esa pieza.

Además, revisamos los números del documento contra los papers originales. Casi todos son correctos, con una excepción que señalamos en su lugar: el *rotation trick* mejora el FID de reconstrucción de VQGAN de 5,0 a 1,1, y no a 1,6 como dice el documento.

La Figura 2 muestra el recorrido. Los capítulos II y III dan las herramientas. El IV reúne la evidencia empírica. El V desarrolla en detalle una de las piezas, el redondeo, y la idea de «seguir el error». El VI intenta atribuir los efectos a las piezas. El VII trata los costos (los distintos tipos de colapso) y su relación con el error acumulado. El VIII revisa la familia JEPA, y el IX condensa todo en criterios para decidir cuándo discretizar.

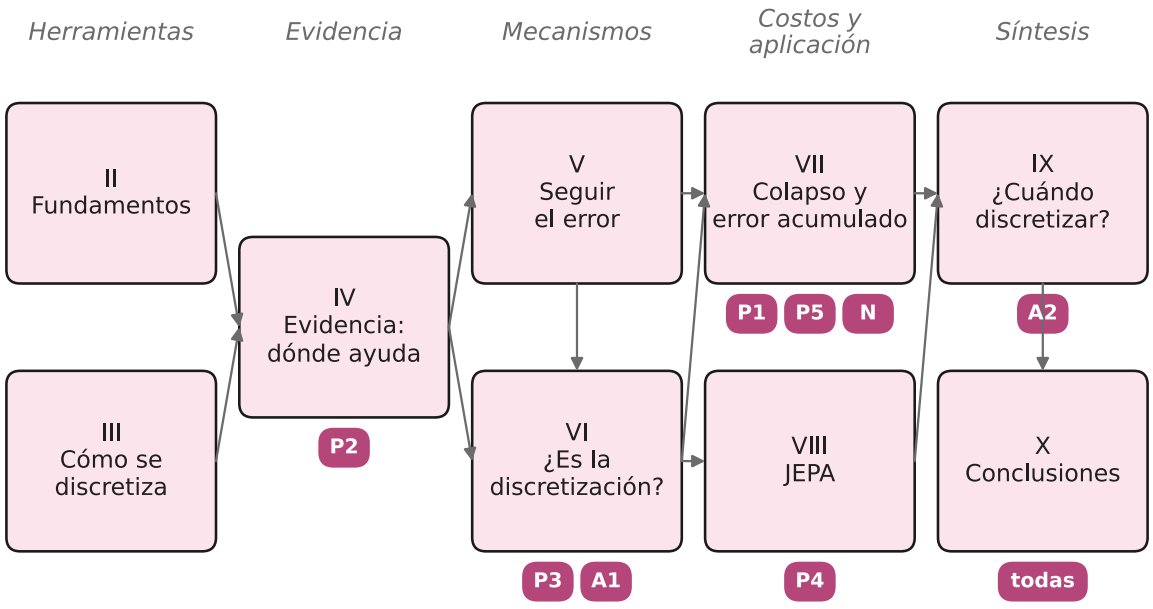


Figura 2. Mapa del capítulo. Debajo de cada capítulo aparecen las preguntas que responde (ver la tabla de la guía de lectura).

Qué nos llevamos

- «Discretizar» cambia al menos seis cosas a la vez: capacidad, geometría, pérdida, elección entre modos, redondeo e identidad exacta.
- La pregunta útil no es «¿discreto o continuo?», sino «¿qué pieza necesita este problema, y se puede obtener sin las demás?».
- El documento del profesor se lee como un catálogo de evidencia; nuestro trabajo es darle un hilo, atribuyendo cada resultado a una pieza.

Capítulo II

Fundamentos

En una frase *Seis ideas bastan para leer el resto: qué es un espacio latente, cuánta información cabe en un código, por qué la regresión promedia, qué es un modelo del mundo, cómo se aprende sin etiquetas y cómo se lee un experimento.*

II.1 Espacio latente

Casi todos los modelos de este capítulo tienen la misma forma (Figura 3). Un **codificador** (*encoder*) toma una observación x (una imagen, un estado de un juego) y la resume en un vector z, llamado **latente**. Un **decodificador** (*decoder*) intenta reconstruir x a partir de z, o bien un **predictor** intenta adivinar el z del instante siguiente. Como z es mucho más pequeño que x, el codificador está obligado a quedarse con lo importante. A esa restricción se le llama **cuello de botella**.

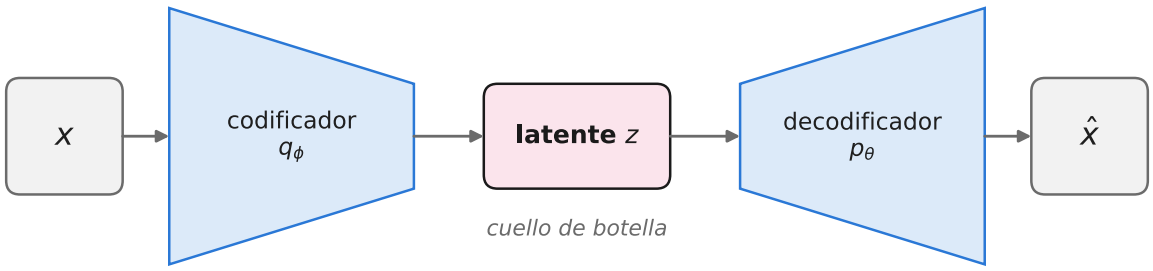


Figura 3. Un autoencoder. La pregunta de este capítulo es qué ocurre cuando el latente z deja de ser un vector cualquiera de $\mathbb{R}^d$ y pasa a ser uno de K códigos.

El VAE (Kingma & Welling, 2014) agrega una capa probabilística: el codificador entrega una distribución $q_\varphi(z|x)$ en vez de un punto, y una pérdida (la divergencia KL con una distribución *a priori* $p(z)$) evita que esa distribución se vuelva arbitrariamente precisa. Lo vimos en detalle en los apuntes. Aquí basta recordar que el KL actúa como un «impuesto» sobre la información que el latente transporta. Volveremos a esto al hablar de colapso posterior (capítulo VII).

II.2 Información: cuánto cabe en un código

> **Idea base** *Bits y entropía*
>
> Un **bit** es la información necesaria para distinguir entre dos alternativas igual de probables. Para distinguir entre K alternativas igual de probables hacen falta $\log_2 K$ bits: un diccionario de 512 códigos lleva como máximo 9 bits por código.
>
> Si las alternativas no son igual de probables, la información promedio es la **entropía**,
>
> $$H = -\sum_{k=1}^{K} p_k \log_2 p_k,$$
>
> que vale $\log_2 K$ cuando todos los códigos se usan por igual y 0 cuando siempre se usa el mismo. La **perplejidad** 2^H se interpreta como el «número efectivo de códigos usados». Un diccionario de 512 códigos con perplejidad 20 se comporta como si tuviera solo 20.

¿Y un vector continuo? En teoría, un solo número real puede codificar infinitos bits (basta con escribirlos en sus decimales). En la práctica, el ruido lo impide: si cada coordenada se transmite con un error de tamaño σ, solo se pueden distinguir de forma confiable unos A/σ niveles en un rango de amplitud A. Shannon (Shannon, 1948) formalizó esto: la capacidad de un canal con ruido es finita y crece con el logaritmo de la razón entre señal y ruido.

Esto importa por una razón concreta. Cuando un paper compara un latente discreto con uno continuo «del mismo tamaño», casi nunca compara la misma cantidad de información. El discreto tiene un techo explícito ($\log_2 K$ bits por código) y el continuo tiene un techo implícito, que depende del ruido, la precisión numérica y la regularización. Es la pieza 1 del paquete, y es el primer confusor que hay que controlar.

II.3 Regresión frente a clasificación

Esta es quizás la idea más importante del capítulo, porque explica de un golpe por qué las predicciones continuas «se ven borrosas» y por qué muchos modelos del mundo pasaron a predecir categorías.

Supongamos que el estado siguiente Y es aleatorio: con probabilidad $1/2$ el agente gira a la izquierda (estado A) y con probabilidad $1/2$ gira a la derecha (estado B). Un predictor entrenado con **error cuadrático medio** (MSE) busca la constante c que minimiza

$$\mathbb{E}\big[(Y-c)^2\big] \quad \Rightarrow \quad c^{*} = \mathbb{E}[Y] = \frac{A+B}{2}.$$

Es decir, el predictor óptimo entrega el **promedio** de los futuros posibles. Si A y B son dos imágenes, el promedio es una superposición borrosa de ambas. Si son dos posiciones, es un punto intermedio que quizás es una pared. En ningún caso es un futuro que pueda ocurrir (Figura 4, izquierda).

Un predictor entrenado con **entropía cruzada** (CE) no entrega un punto, sino una distribución $\hat{p}$ sobre categorías, y minimiza

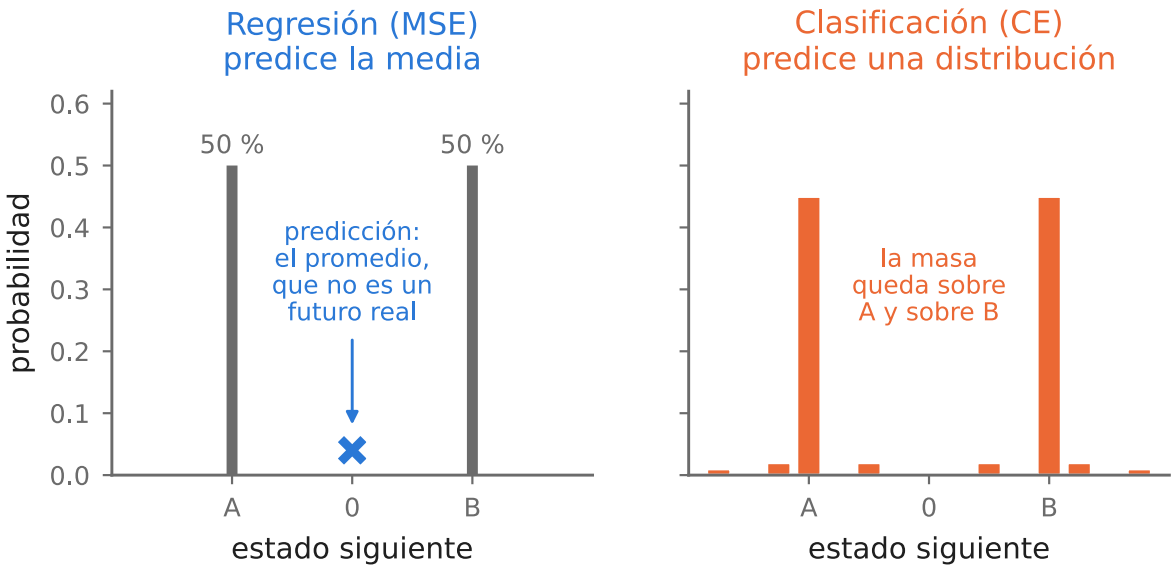


Figura 4. El mismo problema, dos pérdidas. Con MSE, el óptimo es el promedio de los futuros; con entropía cruzada, el óptimo es la distribución completa.

$$\mathbb{E}[-\log \hat{p}(Y)] \quad \Rightarrow \quad \hat{p}^* = p,$$

o sea, la distribución verdadera completa. Si los futuros son A y B con probabilidad $1/2$ cada uno, aprende exactamente eso (Figura 4, derecha). Después se puede muestrear un futuro concreto, o planificar considerando ambos.

Esto no tiene nada que ver con que el **estado** sea discreto: tiene que ver con lo que el predictor está autorizado a entregar. Un predictor de vectores continuos también puede entregar una distribución con varios modos (una mezcla de gaussianas, o un modelo de difusión, como veremos en el capítulo IV). La entropía cruzada sobre categorías es simplemente la forma más barata de conseguirlo. Por eso separamos dos piezas: la ❸ (la pérdida) y la ❹ (poder elegir entre modos).

> **Para pensar** *Si A ocurre con probabilidad $0,9$ y B con $0,1$, ¿el predictor MSE entrega un futuro real? ¿Qué entrega?*

II.4 Aprendizaje por refuerzo y modelos del mundo

> **Idea base** *Aprendizaje por refuerzo en un párrafo*
>
> Un **agente** observa el estado del **entorno**, elige una **acción**, y recibe una **recompensa** y un nuevo estado. Su objetivo es aprender una **política** (una regla para elegir acciones) que maximice la suma de recompensas futuras, llamada **retorno**. Los métodos *sin modelo* (*model-free*) aprenden la política directamente de la experiencia. Los métodos *basados en modelo* (*model-based*) aprenden además un **modelo del mundo**: un simulador aprendido que predice qué estado y qué recompensa vendrán después de cada acción.

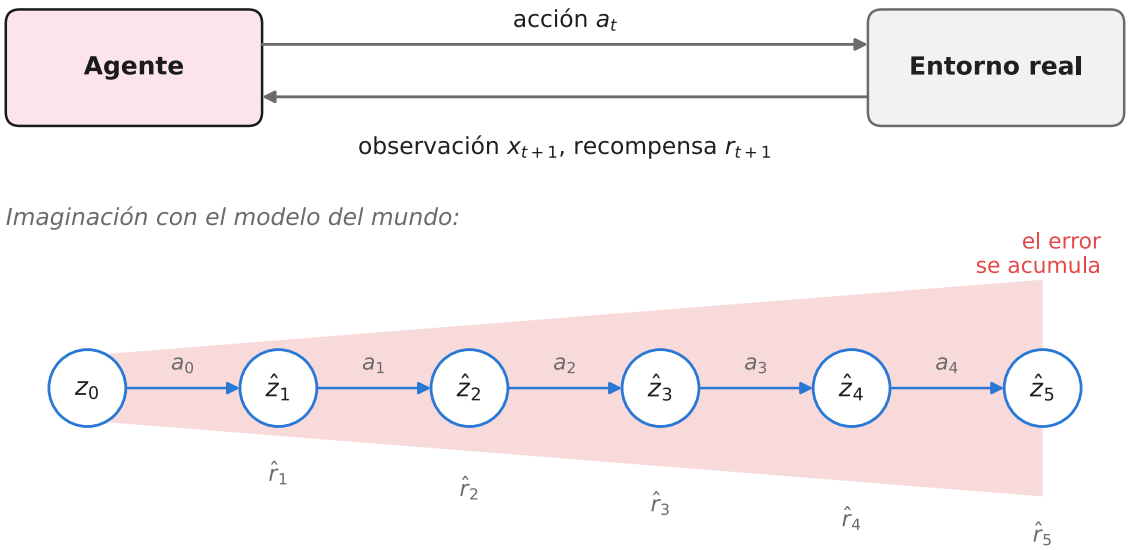


Figura 5. Arriba, el ciclo agente-entorno. Abajo, un rollout imaginado: cada estado predicho $\hat{z}_t$ se usa como entrada para predecir el siguiente, así que los errores se arrastran y crecen.

Un modelo del mundo permite **imaginar**: partir del estado actual, aplicar acciones en la cabeza y ver qué pasaría, sin tocar el entorno real (Figura 5). A una secuencia imaginada se le llama **rollout**. Dreamer, por ejemplo, entrena su política casi exclusivamente con rollouts imaginados en su espacio latente.

El detalle crucial es que en un rollout **la salida de cada paso es la entrada del siguiente**. Un error pequeño en $\hat{z}_1$ se convierte en la entrada de $\hat{z}_2$, que agrega su propio error, y así sucesivamente. Este fenómeno, el **error compuesto**, es la razón por la que la pieza 5 (redondeo) importa tanto en modelos del mundo, y es el tema del capítulo V.

II.5 Aprender sin etiquetas

> **Idea base** *Aprendizaje autosupervisado*
>
> En el aprendizaje **autosupervisado** el modelo se inventa su propia tarea a partir de datos sin etiquetar. Hay dos familias principales.
> - **Contrastiva:** se generan dos versiones de una misma imagen (por ejemplo, recortes distintos) y se entrena para que sus representaciones queden cerca entre sí y lejos de las de otras imágenes.
> - **Predictiva:** se oculta una parte de la entrada y se entrena para predecir la parte oculta, ya sea en píxeles (MAE) o en el espacio de representaciones (JEPA).

El riesgo central de la familia predictiva en espacio de representaciones es una solución trampa: si el codificador entrega **el mismo vector para todo**, predecir la representación de la parte oculta es trivial y la pérdida vale cero. A esto se le llama **colapso de representaciones**. BYOL (Grill et al., 2020) e I-JEPA (Assran et al., 2023) lo evitan con dos trucos. El primero es que el objetivo lo calcula un **segundo codificador** que no se entrena por gradiente, sino que se actualiza como un promedio

móvil exponencial (*EMA*) del primero. El segundo es que el gradiente no fluye hacia ese codificador objetivo (*stop-gradient*). Así, el objetivo cambia lentamente y el modelo no puede «moverlo» hacia la solución trivial. Volveremos a esto en los capítulos VII y VIII, porque el colapso es uno de los costos que la discretización transforma.

II.6 Cómo leer un resultado experimental

Idea base *Vocabulario experimental*

- **Baseline:** el método con el que se compara.
- **Ablación:** quitar o cambiar una sola pieza del método y medir cuánto empeora. Es la herramienta principal para atribuir mejoras a piezas.
- **Benchmark:** conjunto estándar de tareas. Atari son 55 videojuegos; DMControl y Meta-World son tareas de control de robots simulados; Minigrid son laberintos en grilla.
- **Puntaje normalizado por humanos (Atari):** $(\text{agente} - \text{azar})/(\text{humano} - \text{azar})$. Vale 1 si el agente juega como un jugador humano de referencia.
- **FID** (Heusel et al., 2017): distancia entre la distribución de imágenes generadas y la de imágenes reales, medida con las características de una red preentrenada. **Menor es mejor**. El **rFID** aplica la misma medida a imágenes reconstruidas por un autoencoder: mide cuánto se pierde al comprimir.
- **Uso del diccionario:** fracción de los códigos que efectivamente se usan, o su perplejidad.

Un consejo general, que el .epub ya daba y que aquí se vuelve operativo: ante cada resultado, preguntarse **qué cambió además de la discretización**. Si la respuesta es «varias cosas», el resultado es evidencia de que el paquete ayuda, no de que la discreción ayude.

Qué nos llevamos

- Un código de K entradas lleva como máximo $\log_2 K$ bits; un vector continuo tiene un techo implícito que depende del ruido. Comparar «a igual tamaño» no es comparar a igual información.
- Con MSE, el predictor óptimo es el **promedio** de los futuros; con entropía cruzada, es la **distribución**. Esto es independiente de que el estado sea discreto.
- En un rollout, la salida de cada paso es la entrada del siguiente, así que los errores se arrastran.
- Los métodos predictivos autosupervisados pueden colapsar a una constante; EMA y *stop-gradient* son las defensas estándar.

Capítulo III

Cómo se discretiza un espacio continuo

En una frase *Hay varias formas de discretizar los valores de un latente (one-hot, VQ, FSQ, residual, binaria) y varios niveles en los que se puede discretizar (el latente, su estructura, el predictor, la pérdida, las acciones); todas pagan el mismo precio: el gradiente no atraviesa un redondeo.*

III.1 Cuatro maneras de discretizar un vector

Partimos de un vector continuo $z_e \in \mathbb{R}^d$, la salida del codificador. Discretizar es elegir una función q que lo lleve a un conjunto finito de valores. La Figura 6 muestra las tres geometrías principales en dos dimensiones.

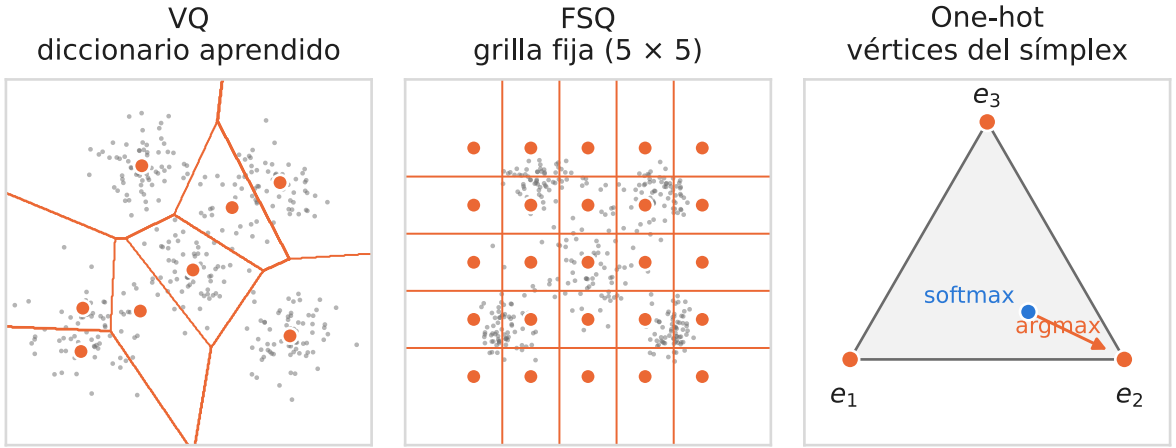


Figura 6. Tres formas de partir el plano. En VQ, los centros (naranjos) se aprenden y cada uno se queda con la región de puntos más cercanos (celdas de Voronoi). En FSQ, la grilla es fija y no se aprende nada. En *one-hot*, los códigos son los vértices de un símplex: el modelo produce una distribución (un punto interior, en azul) y se queda con el vértice más probable.

One-hot categórico

El codificador produce K números (*logits*), los convierte en probabilidades con un *softmax* y se queda con la categoría más probable, o muestrea una. El resultado se representa como un vector *one-hot*:

ceros en todas partes y un 1 en la posición elegida. Para muestrear y aun así propagar gradientes se usa la Gumbel-Softmax (Jang et al., 2017), que vimos en los apuntes: se agrega ruido de Gumbel a los logits y se aplica un softmax con temperatura τ, que al bajar hace la muestra cada vez más parecida a un *one-hot*.

Una sola variable categórica de K clases lleva poca información. Por eso DreamerV2 usa **32 variables categóricas de 32 clases cada una**: el latente es un vector binario de largo 1024 con exactamente 32 unos. Esa estructura, muchas categorías pequeñas en paralelo, aparece una y otra vez.

VQ: el vecino más cercano en un diccionario

> **Idea base** *k-means*
>
> Dado un conjunto de puntos, el algoritmo **k-means** busca K centros que los representen bien. Alterna dos pasos: asignar cada punto a su centro más cercano y mover cada centro al promedio de sus puntos. La región de puntos que quedan más cerca de un centro que de cualquier otro se llama su **celda de Voronoi**.

La cuantización vectorial (VQ) del VQ-VAE (Oord et al., 2017) es un k-means que se aprende junto con la red. Hay un **diccionario** (*codebook*) de K vectores $e_1, \dots, e_K \in \mathbb{R}^d$, y cada salida del codificador se reemplaza por el más cercano:

$$z_q = e_k, \quad k = \arg\min_j \big\| z_e - e_j \big\|.$$

La pérdida tiene tres términos:

$$\mathcal{L} = \underbrace{-\log p\big(x \mid z_q\big)}_{\text{reconstruir}} + \underbrace{\big\| \mathrm{sg}[z_e] - e_k \big\|^2}_{\text{mover el código}} + \beta \underbrace{\big\| z_e - \mathrm{sg}[e_k] \big\|^2}_{\text{compromiso}},$$

donde $\mathrm{sg}[\cdot]$ (*stop-gradient*) significa «trata esto como constante». El segundo término acerca el código elegido a la salida del codificador, como el paso de promedio de k-means. El tercero, la pérdida de **compromiso** (*commitment*), impide que el codificador se aleje indefinidamente de los códigos.

FSQ: redondear cada coordenada

FSQ (*finite scalar quantization*, Mentzer et al. (2024)) elimina el diccionario aprendido. Cada coordenada de z_e se acota con una función como tanh y se redondea a uno de L_i niveles fijos:

$$\hat{z}_i = \mathrm{round}\big(\lfloor L_i/2 \rfloor \cdot \tanh(z_i)\big)$$

(esta es la versión para L_i impar; para L_i par se agrega un pequeño desplazamiento). El redondeo se atraviesa con el estimador *straight-through* que veremos en la Sección III.4. El conjunto de códigos es la grilla producto, con $K = \prod_i L_i$ elementos. Por ejemplo, con niveles $(8, 5, 5, 5)$ hay $8 \cdot 5 \cdot 5 \cdot 5 = 1000$ códigos. No hay términos de pérdida extra ni códigos que aprender: todos los códigos existen desde el principio y están igual de separados. Veremos en el capítulo VII que eso cambia radicalmente el problema del colapso del diccionario.

Cuantización residual y binaria

Dos variantes completan el panorama.

- **Cuantización residual (RVQ)**: se cuantiza z, se calcula el error que quedó (el residuo), se cuantiza ese residuo con un segundo diccionario, y así M veces. El código final es la suma $e^{(1)}_{k_1} + \ldots + e^{(M)}_{k_M}$. Con M diccionarios de K entradas se obtienen K^M combinaciones guardando solo MK vectores. Es el estándar en compresión de audio, por ejemplo en SoundStream (Zeghidour et al., 2021).
- **Cuantización binaria (LFQ)**: cada coordenada se reemplaza por su signo, así que un vector de d coordenadas da 2^d códigos sin diccionario (Yu et al., 2024). Es el caso extremo de FSQ, con dos niveles por coordenada.

III.2 Otros niveles donde se puede discretizar

Hasta aquí discretizamos los **valores** del latente. Pero los papers discretizan cosas muy distintas, y conviene tenerlas separadas desde ya (Figura 7).

- **La estructura del latente**. JEPA-Anything (Cui et al., 2026) no discretiza valores: parte el espacio de representación en K factores que no se solapan, cada uno con su predictor. Lo que se vuelve discreto es **cuántas piezas hay y cuál es cuál**.
- **La elección del predictor**. Un modelo puede tener K predictores continuos y elegir uno, como MoP-JEPA, o predecir un «código de azar» discreto que indica qué rama del futuro ocurrió, como Stochastic MuZero. El estado sigue siendo continuo; lo discreto es la decisión.

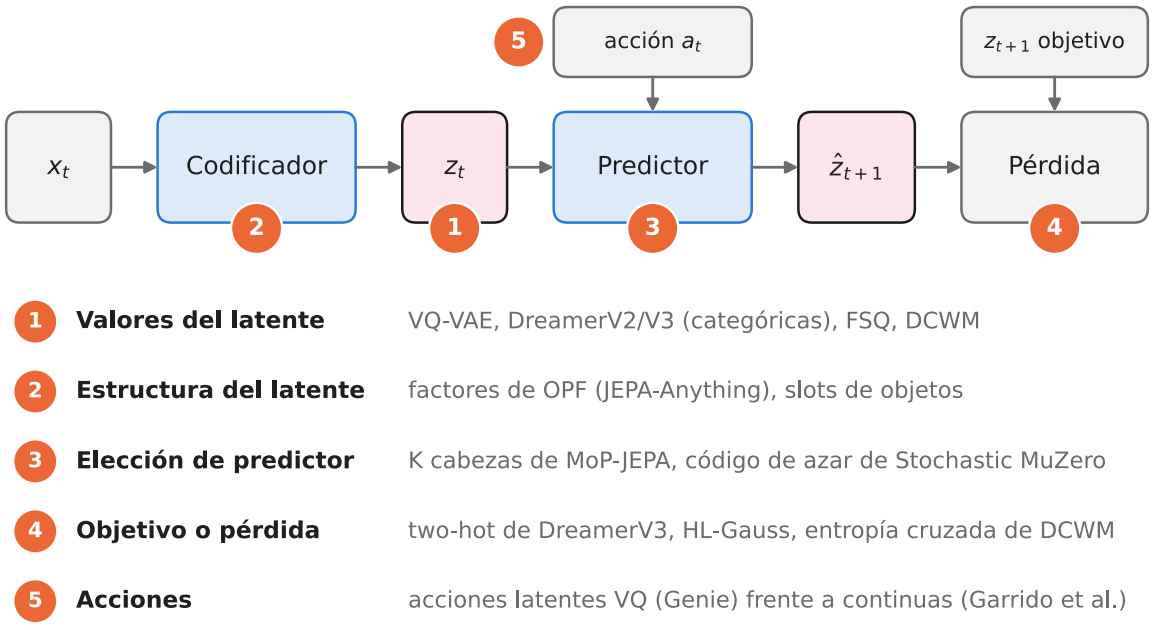


Figura 7. Cinco lugares de un modelo del mundo donde se puede discretizar. Los números se usan en el resto del capítulo para ubicar cada paper.

- **El objetivo de la pérdida**. Un número real y (una recompensa, un valor) se puede predecir por regresión o representar como una distribución sobre intervalos y predecir por clasificación. La versión más simple es **two-hot**: si y cae entre dos intervalos vecinos b_i y b_{i+1}, se reparte el peso entre ambos en proporción a la cercanía, y la predicción es el promedio ponderado. DreamerV3 lo usa para recompensas y valores (Hafner et al., 2025) HL-Gauss (Imani & White, 2018) suaviza el objetivo con una gaussiana. Farebrother et al. (2024) muestran que entrenar funciones de valor por clasificación en vez de regresión mejora el aprendizaje por refuerzo profundo en muchos dominios. **El estado no se discretiza; solo la pérdida.**
- **Las acciones**. Los modelos que aprenden acciones a partir de videos sin etiquetar deben inventar un «vocabulario» de acciones. Pueden hacerlo discreto (Genie) o continuo (Garrido et al.). Lo vemos en el capítulo IV.

Esta lista ya responde parte de P4 (¿alguien discretiza en JEPA?): sí, pero casi nunca los valores del estado. Lo retomamos en el capítulo VIII.

III.3 One-hot o vector: la misma información, distinta geometría

Supongamos que ya tenemos un código k. ¿Cómo se lo entregamos a la red siguiente? Hay dos opciones, y la Figura 8 muestra por qué no son equivalentes.

- Como **one-hot**, todos los códigos quedan a la misma distancia y son ortogonales entre sí. No hay noción de parecido. Eso es malo si las categorías tienen un orden natural (posiciones, intensidades), pero tiene una ventaja poco obvia: como los vectores son ortogonales, ajustar lo que la red hace con el código 3 **no afecta** lo que hace con el código 7. Hay muy poca **interferencia** entre categorías.
- Como **vector del diccionario** e_k, los códigos parecidos quedan cerca. La red puede generalizar de un código a sus vecinos, pero también puede confundirlos, y los cambios en uno afectan a los cercanos.

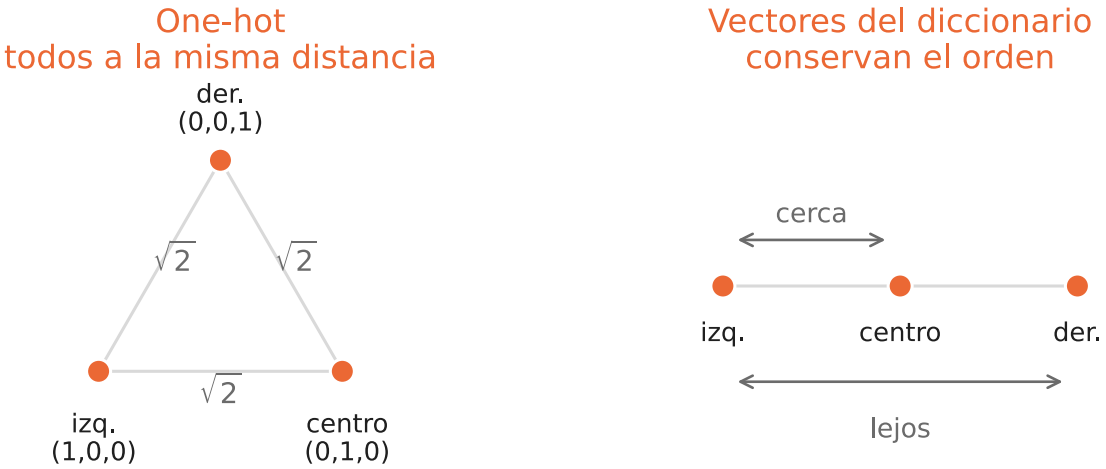


Figura 8. Tres posiciones de un robot (izquierda, centro, derecha). Como *one-hot*, las tres quedan a la misma distancia ($\sqrt{2}$) y el modelo no sabe que «centro» está entre las otras dos. Como vectores del diccionario, la cercanía conserva el orden.

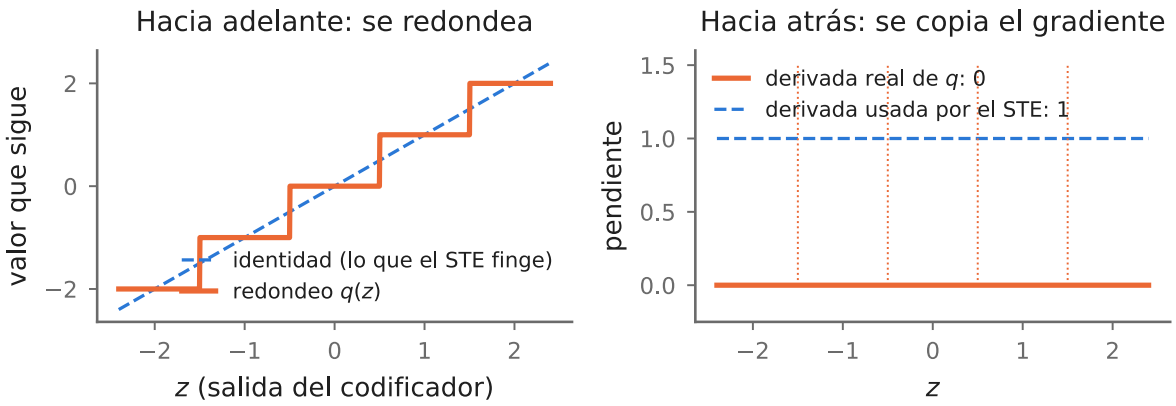


Figura 9. El estimador *straight-through* (STE). Hacia adelante se usa el valor redondeado (naranjo). Hacia atrás se finge que la función era la identidad (azul) y se copia el gradiente sin cambios.

Esta es la pieza ❷. En el capítulo IV veremos dos papers que la miden directamente y llegan a conclusiones opuestas: en laberintos, gana *one-hot*; en control continuo, ganan los vectores. No hay contradicción. Gana la geometría que coincide con la estructura del problema.

> **Para pensar** *Las letras de un alfabeto, ¿se parecen más a un* one-hot *o a vectores con distancia? ¿Y los colores?*

III.4 El precio: cómo pasar el gradiente

La función de redondeo es una escalera: es plana en casi todas partes y salta en los bordes. Su derivada es cero en casi todas partes. Si se aplica la regla de la cadena tal cual, el codificador no recibe ninguna señal de entrenamiento (Figura 9).

La solución estándar es el **estimador straight-through** (STE, Bengio et al. (2013)), que en código se escribe con un truco:

$$z_q = z_e + \mathrm{sg}[q(z_e) - z_e].$$

Hacia adelante, $z_q = q(z_e)$, el valor redondeado. Hacia atrás, el término con sg no aporta gradiente, así que $\partial z_q / \partial z_e = 1$: el gradiente pasa intacto. El problema es que ese gradiente es **el que habría tenido el punto sin redondear**, no el del código que realmente se usó. La diferencia entre ambos es un sesgo que nadie corrige.

¿Cuánto cuesta ese sesgo? El *rotation trick* (Fifty et al., 2025) da una medida. En vez de copiar el gradiente, transforma z_e en e_k con una rotación y un cambio de escala que se tratan como constantes al derivar, de modo que el gradiente que recibe el codificador «sabe» en qué ángulo y a qué distancia estaba del código. Sin cambiar nada más, mejora la reconstrucción, el uso del diccionario y el error de cuantización en los 11 esquemas de entrenamiento de VQ-VAE que prueban. En el caso más llamativo, un VQGAN (Esser et al., 2021) con 16.384 códigos sobre ImageNet, el rFID baja de 5,0 a 1,1 y el uso del diccionario sube del 2 % al 27 % ESTABLECIDO. (El documento del profesor reporta 1,6; la versión actual del paper dice 1,1.)

La Gumbel-Softmax enfrenta el mismo problema con otro compromiso. A temperatura alta, el gradiente es poco sesgado pero la muestra es poco discreta; a temperatura baja, la muestra es casi *one-hot* pero el gradiente es muy ruidoso. Por eso NLI va bajando τ durante el entrenamiento.

Este costo de optimización no es una pieza del paquete en el sentido de la Figura 1, porque no explica ninguna ventaja. Es el precio de entrada que todo método discreto paga, y la razón por la que muchos resultados discretos mejoran solo con cambiar el estimador.

Qué nos llevamos

- Las formas principales de discretizar valores son *one-hot* (DreamerV2), VQ (VQ-VAE), FSQ (grilla fija), RVQ (residual) y LFQ (binaria). Difieren en si el diccionario se aprende y en su geometría.
- Además de los valores, se puede discretizar la estructura, la elección de predictor, la pérdida o las acciones. Muchos «modelos discretos» solo discretizan una de estas cosas.
- *One-hot* y vector del diccionario llevan la misma información con distinta geometría: *one-hot* evita la interferencia, el vector conserva la cercanía.
- El gradiente no atraviesa un redondeo. El STE lo copia con sesgo, y ese sesgo cuesta: el *rotation trick* baja el rFID de un VQGAN de 5,0 a 1,1 sin tocar nada más ESTABLECIDO.

Capítulo IV

Evidencia: dónde ayuda y dónde no

En una frase *La idea de que discretizar ayuda nació en los modelos del mundo para aprendizaje por refuerzo; los experimentos más cuidadosos muestran que la ventaja es real pero condicional (a la capacidad, a la estocasticidad, a la geometría), y hay dominios donde lo continuo gana con claridad.*

Este capítulo responde a P2. Recorre los resultados que construyeron la idea de que «lo discreto es mejor», en orden aproximadamente cronológico, y termina con los casos donde lo continuo gana. Cada modelo se presenta con el mismo formato: qué hace, qué discretiza, qué mejora y qué empeora, y a qué pieza del paquete se puede atribuir el efecto.

IV.1 La línea de Dreamer: de gaussiano a categórico

PlaNet y el modelo de estado recurrente

> **Fuente.** *«Learning Latent Dynamics for Planning from Pixels»* de D. Hafner et al. (2019). [arXiv:1811.04551]

PlaNet (Hafner et al., 2019) aprende un modelo del mundo a partir de imágenes y planifica dentro de él. Su pieza central, que heredan todos los Dreamer, es el **modelo de estado recurrente** (RSSM, Figura 10). El estado del mundo en el instante t tiene dos partes:

- una parte **determinista** h_t, que es la memoria de una red recurrente y se actualiza con el estado y la acción anteriores;
- una parte **estocástica** z_t, que se muestrea de una distribución que depende de h_t y que absorbe lo impredecible.

Durante el entrenamiento, z_t se calcula mirando la imagen actual (la *posterior*). Durante la imaginación no hay imagen, así que se usa una predicción de z_t hecha solo desde h_t (la *prior*). El entrenamiento empuja la prior hacia la posterior con una pérdida KL, igual que en un VAE.

PlaNet mostró que ambas partes son necesarias: un modelo puramente determinista y uno puramente estocástico planifican peor que el RSSM. Es un primer dato para la pregunta A1. **La memoria continua y la elección aleatoria cumplen funciones distintas**, y el diseño que funciona tiene las dos.

DreamerV2: el cambio a categóricas

> **Fuente.** *«Mastering Atari with Discrete World Models»* de D. Hafner et al. (2021). [arXiv:2010.02193]

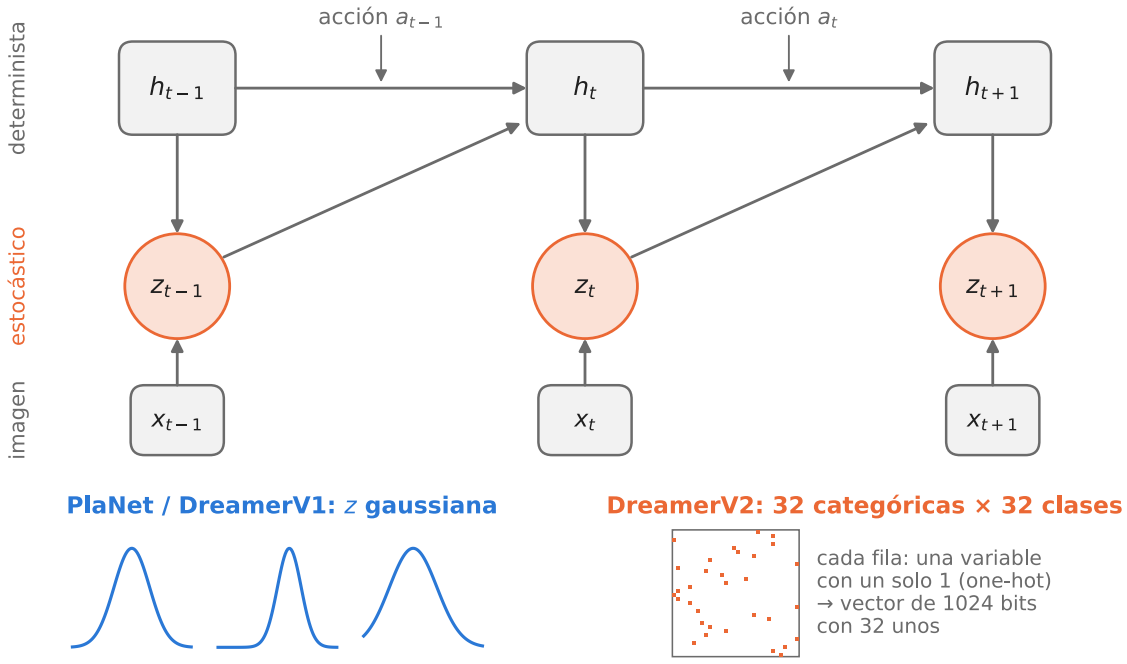


Figura 10. El modelo de estado recurrente (RSSM). Arriba, la cadena en el tiempo: h es la memoria determinista y z la parte estocástica. Abajo, lo único que cambió de PlaNet a DreamerV2: z pasó de ser una gaussiana a ser 32 variables categóricas de 32 clases.

DreamerV2 (Hafner et al., 2021) fue el primer agente que alcanzó nivel humano en Atari (55 juegos, 200 millones de cuadros) aprendiendo su comportamiento **solo dentro de su modelo del mundo**, con una sola GPU. Respecto de su antecesor, hizo pocos cambios. El más visible fue reemplazar la z_t gaussiana por un vector de 32 variables categóricas de 32 clases, entrenadas con el estimador *straight-through*. El otro fue el **KL balancing**: la pérdida KL se reparte de modo que la prior se mueva hacia la posterior más rápido (con peso 0,8) que la posterior hacia la prior (con peso 0,2). Así se evita que las representaciones se regularicen hacia una prior que todavía es mala.

La ablación es clara: las variables categóricas superan a las gaussianas en 42 juegos, pierden en 8 y empatan en 5 (empate significa una diferencia menor al 5 %) ESTABLECIDO. En la mediana del puntaje normalizado por humanos, el agente completo obtiene 1,64 y la versión sin latentes discretos, 1,08. El KL balancing gana en 44 juegos, pierde en 6 y empata en 5.

Lo más valioso para nosotros es lo que los autores dicen **no saber**. Escriben que desconocen por qué las categóricas ayudan, y proponen cuatro hipótesis para investigar:

1. Una prior categórica puede ajustar exactamente la «posterior agregada» (el promedio de las posteriores de todas las imágenes), porque una mezcla de categóricas sigue siendo categórica. Una gaussiana no puede imitar una mezcla de gaussianas. Esto sería la pieza 4, **elegir entre modos**.
2. La esparsidad: el latente es un vector binario de 1024 posiciones con solo 32 unos, lo que podría ayudar a generalizar. Esto sería la pieza 2, **geometría**.

3. Las categóricas con *straight-through* podrían ser **más fáciles de optimizar**, porque el estimador ignora un término que de otro modo escalaría el gradiente y podría hacerlo explotar o desvanecerse. Esto no es una ventaja de la representación, sino del estimador.
4. Las categóricas podrían ser un mejor sesgo inductivo para los aspectos **no suaves** de Atari: entrar a una habitación nueva, o que un objeto desaparezca al recogerlo. El mundo de Atari tiene «saltos», y una representación con saltos los modela mejor.

> Para pensar *¿Qué experimento separaría la hipótesis 1 de la 2? Pista: una de ellas debería desaparecer en un juego sin azar.*

La ablación de DreamerV2 cambia a la vez la discreción, la esparsidad, el estimador del gradiente y la capacidad de modelar varios modos. Por eso no permite elegir entre las cuatro hipótesis. Es un ejemplo perfecto de evidencia de que **el paquete** ayuda.

DreamerV3: discretizar también la pérdida

DreamerV3 (Hafner et al., 2025) generalizó Dreamer a más de 150 tareas con un único conjunto de hiperparámetros. Conservó las categóricas y les agregó ajustes que conviene conocer, porque cada uno ataca un problema concreto:

- **Unimix**: cada distribución categórica se mezcla con un 1 % de distribución uniforme. Así ninguna probabilidad llega a ser exactamente cero, y el KL no explota cuando la posterior elige una clase que la prior consideraba imposible.
- **Free bits**: la pérdida KL no se penaliza por debajo de 1 nat. Esto impide que el modelo «apague» el latente para ahorrarse el KL, que es una forma de colapso posterior (capítulo VII).
- **Symlog y two-hot**: las recompensas y los valores se predicen por **clasificación** sobre 255 intervalos, con objetivos *two-hot*, en vez de por regresión. Es la pieza 3 aplicada a la pérdida, sin tocar el estado.

Qué se discretiza	La parte estocástica del estado, z_t (32 × 32 categóricas). En V3, además, el objetivo de recompensas y valores (two-hot).
Qué mejora	Puntaje en Atari: 42 de 55 juegos mejoran respecto de z_t gaussiana ESTABLECIDO.
Qué empeora o cuesta	Necesita *straight-through*, unimix y free bits para entrenar de forma estable.
Pieza del paquete	No identificada. Los autores proponen 4, 2, el estimador y los «saltos» del mundo, sin poder separarlos.

IV.2 El experimento más limpio: Meyer et al.

> Fuente. *«Harnessing Discrete Representations for Continual Reinforcement Learning»* de E. Meyer, A. White y M. C. Machado (2024). [arXiv:2312.01203]

Si hubiera que leer un solo paper sobre nuestra pregunta, sería este (Meyer et al., 2024). Ya está en la carpeta `Additional` del repositorio. Su valor no está en un gran resultado, sino en el diseño: cambia **una cosa a la vez**.

> **Idea base** *Minigrid y aprendizaje continuo*
>
> **Minigrid** es una colección de laberintos en grilla. El agente (un triángulo rojo) ve una imagen pequeña del laberinto y puede girar, avanzar y, en algunas versiones, recoger una llave y abrir una puerta. En el **aprendizaje por refuerzo continuo** (*continual RL*), el entorno cambia cada cierto tiempo (aquí, la posición de las paredes y objetos), y lo que importa es cuán rápido se **readapta** el agente, no cuán bien juega en un entorno fijo.

El diseño

El modelo del mundo se divide en dos partes: un **codificador** que transforma la imagen en un latente, y un **modelo de transición** (una red pequeña, fija) que predice el latente siguiente a partir del actual y de la acción. Los autores cambian solo el codificador y comparan tres tipos de latente (Figura 11):

- **continuo denso**, de un autoencoder común;
- **continuo esparso**, del mismo autoencoder con una activación (FTA, Pan et al. (2021)) que deja casi todas las coordenadas en cero;
- **discreto multi-one-hot**, de un VQ-VAE cuyos códigos se entregan como varios vectores *one--hot*.

Los entornos tienen un 10 % de probabilidad de que la acción elegida se reemplace por otra al azar, así que el futuro es estocástico. Para medir la calidad del modelo no se compara un estado predicho con el real (puede haber varios futuros válidos), sino la **distribución** de estados que el modelo genera en rollouts de 30 pasos con la que genera el entorno real, mediante una divergencia KL.

Los resultados

Primero, el modelo discreto es más preciso, y la diferencia crece con la complejidad del entorno. En el laberinto vacío no hay diferencia. En los laberintos con una abertura en la pared o con llave y

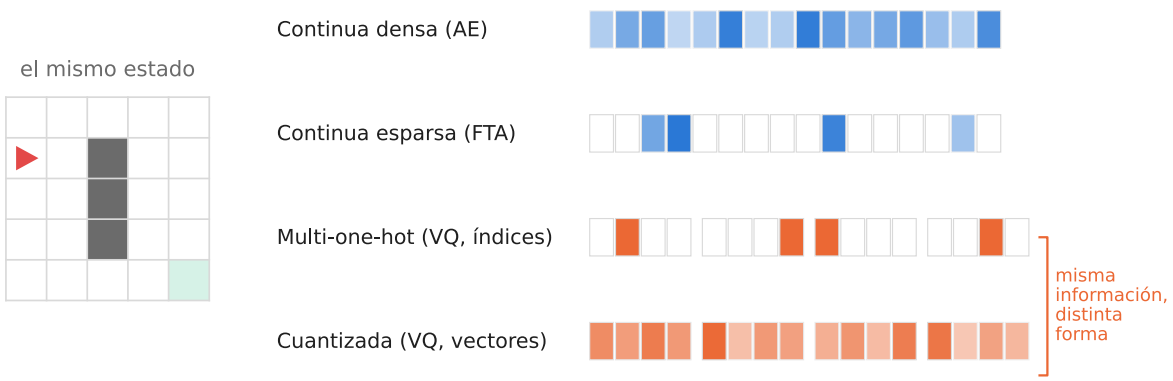


Figura 11. El mismo estado de un laberinto, representado de cuatro formas. Las dos últimas salen del mismo VQ-VAE y llevan exactamente la misma información, pero una la entrega como índices *one-hot* y la otra como los vectores del diccionario.

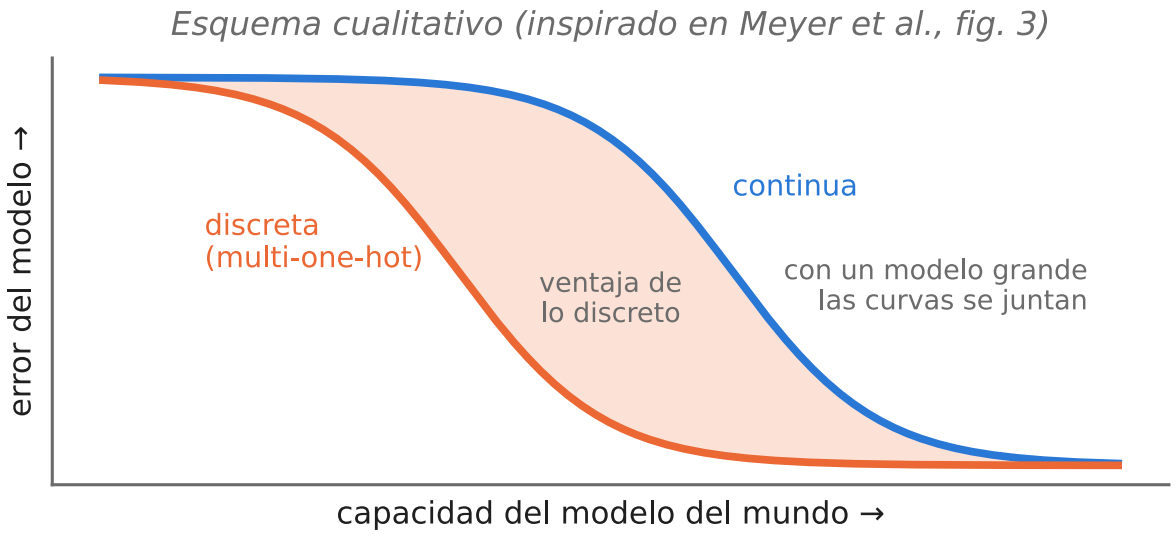


Figura 12. Esquema cualitativo del resultado de escalamiento de Meyer et al.: la ventaja de la representación discreta aparece solo cuando el modelo del mundo tiene poca capacidad en relación con la complejidad del entorno.

puerta, los modelos continuos casi nunca predicen que el agente cruce la abertura o recoja la llave. Son las transiciones **raras** en los datos de entrenamiento ESTABLECIDO.

Segundo, la ventaja depende de la capacidad. Al agrandar el modelo de transición, las curvas de todos los métodos se juntan: con un modelo suficientemente grande, la representación da igual (Figura 12) ESTABLECIDO. Lo discreto ayuda cuando **el mundo es más grande que el modelo**.

Tercero, y es el resultado clave: los autores toman el **mismo** VQ-VAE y entregan sus códigos de dos formas: como índices *one-hot* o como los vectores del diccionario («cuantizados»). Ambas representaciones son discretas y llevan exactamente la misma información. Aun así, la versión *one-hot* es **mucho** mejor. Su conclusión es que el éxito se debe a la naturaleza **binaria y esparsa** de la representación, y no a que sea discreta ESTABLECIDO.

Hay un matiz que conviene notar. En ese experimento, el modelo sobre vectores se entrenó con error cuadrático y el modelo *one-hot* con entropía cruzada, así que también cambió la pieza [3]. Los autores reportan en una nota que entrenar el modelo discreto con error cuadrático «hizo poca diferencia», lo que reduce la preocupación, pero no la elimina.

Cuarto, en aprendizaje continuo, donde el laberinto se reordena cada 40 000 pasos (o cada 100 000 en el de llave y puerta), la recompensa promedio por configuración es:

Latente	Abertura en la pared	Llave y puerta
De extremo a extremo (sin autoencoder)	28 ± 5	14 ± 2
Continuo denso	382 ± 33	866 ± 94
Continuo esparso (FTA)	574 ± 57	1033 ± 130
Discreto (VQ-VAE, *one-hot*)	**674 ± 21**	**1324 ± 64**

El orden es discreto, después esparso continuo y después denso continuo ESTABLECIDO. Cuando el entorno cambia, el error de reconstrucción del VQ-VAE sube mucho menos que el de los demás: su representación **generaliza mejor a la configuración nueva**.

Un detalle honesto: en el aprendizaje por refuerzo episódico, sin cambios de entorno, el VQ-VAE fue al principio el **peor**, porque su autoencoder aprende más lento. Cuando se espera a que todos los autoencoders estén igual de entrenados, el VQ-VAE y el esparso convergen con dos a tres veces menos actualizaciones de la política que el denso. La ventaja discreta tiene un costo inicial.

Qué se discretiza	Los valores del latente (VQ-VAE), entregados como multi-one-hot.
Qué mejora	Modelos del mundo más precisos con poca capacidad, sobre todo en transiciones raras; readaptación más rápida en aprendizaje continuo ESTABLECIDO.
Qué empeora o cuesta	Aprendizaje inicial más lento. La ventaja desaparece con modelos grandes.
Pieza del paquete	2 (geometría binaria y esparsa: el mismo código como vector rinde mucho peor) y 1 (la ventaja solo aparece con poca capacidad).

IV.3 Control continuo: DCWM

Fuente. *«Discrete Codebook World Models for Continuous Control»* de A. Scannell et al. (2025). [arXiv:2503.00653]

El resultado de Meyer es en laberintos, un mundo naturalmente discreto. ¿Qué pasa en control continuo, donde el estado es una lista de ángulos y velocidades?

> **Idea base** *Control predictivo y MPPI*
>
> En el **control predictivo por modelo** (MPC), en cada instante el agente simula en su modelo del mundo muchas secuencias de acciones posibles, evalúa cuánta recompensa daría cada una, ejecuta **solo la primera acción** de la mejor y repite todo en el instante siguiente. **MPPI** es una forma de hacer esa búsqueda: muestrea secuencias al azar, las promedia ponderándolas por su recompensa y vuelve a muestrear alrededor del promedio. **TD-MPC2** (Hansen et al., 2024) es un modelo del mundo de referencia para control continuo: no reconstruye imágenes, predice su propio latente siguiente con error cuadrático y planifica con MPPI.

DCWM (Scannell et al., 2025) es, en esencia, un TD-MPC2 con el latente discretizado. El codificador usa FSQ, con niveles pequeños ($5 \times 3 = 15$ códigos por símbolo). El modelo de transición **clasifica** el código siguiente con entropía cruzada, en vez de regresionar un vector. No reconstruye observaciones. Se evalúa en 30 tareas de DMControl (incluidos humanoides y perros simulados), 45 de Meta-World (manipulación con un brazo robótico) y 5 de MyoSuite (control de músculos), y es competitivo con TD-MPC2 y DreamerV3.

Lo más útil son sus ablaciones, porque **desarman el paquete** en parte:

- Los latentes discretos superan con claridad a los continuos en eficiencia de muestras ESTABLECIDO.
- Un latente discreto **determinista** entrenado con MSE no alcanza al discreto estocástico entrenado con entropía cruzada. Si el discreto determinista se entrena por clasificación, mejora

respecto del MSE. Los autores concluyen que parte de la ganancia viene de la **estocasticidad** (muestrear el código siguiente) y parte de la **clasificación** ESTABLECIDO. Son las piezas 4 y 3, separadas.

- Los latentes continuos estocásticos (gaussianos o mezclas de gaussianas) ayudan a veces en DMControl, pero rinden mucho peor en Meta-World.
- Sobre la geometría: entregar los códigos como **vectores del diccionario** funciona mejor que como *one-hot* o como etiquetas enteras. La etiqueta entera «no puede modelar la estructura ordinal multidimensional del diccionario», según los autores. El *one-hot* alcanza una eficiencia parecida, pero entrena más lento y, usado dentro del modelo de transición, hace que el entrenamiento se estanque.

El contraste con Meyer es instructivo. En un laberinto, las posiciones de la llave o la puerta son **categorías sin orden**, y gana *one-hot*. En control continuo, los códigos de FSQ son niveles **ordenados** de ángulos y velocidades, y gana el vector, que conserva el orden. La pieza 2 no tiene un ganador universal: **gana la geometría que coincide con la del mundo**.

Qué se discretiza	Los valores del latente (FSQ), con transición por clasificación.
Qué mejora	Eficiencia de muestras frente a latentes continuos; competitivo con TD-MPC2 y DreamerV3.
Qué empeora o cuesta	*One-hot* y etiquetas enteras rinden peor que vectores; hay que elegir la codificación.
Pieza del paquete	4 (estocasticidad) y 3 (clasificación), separadas por ablación; 2 a favor de vectores ordenados.

IV.4 Ramificaciones: elegir en vez de promediar

Volvamos al ejemplo de la Figura 4: si el futuro puede ser A o B, un predictor por regresión entrega su promedio. Dos trabajos atacan ese problema directamente, y los dos discretizan **la elección**, no el estado.

Stochastic MuZero

> **Idea base** *MCTS y MuZero*
>
> La **búsqueda en árbol de Monte Carlo** (MCTS) planifica construyendo un árbol de posibles jugadas: desde el estado actual prueba acciones, simula, evalúa y concentra la búsqueda en las ramas prometedoras. **MuZero** (Schrittwieser et al., 2020) hace MCTS dentro de un modelo aprendido: no conoce las reglas del juego, sino que aprende a predecir el siguiente estado latente, la recompensa y el valor. Su modelo es **determinista**, así que en juegos con azar no tiene cómo representar que una misma jugada puede llevar a resultados distintos.

Stochastic MuZero (Antonoglou et al., 2022) separa cada paso en dos. Primero, la acción del agente lleva a un **estado posterior** (*afterstate*): cómo queda el tablero antes de que actúe el azar. Después, el azar elige un **resultado**, que se representa con un **código discreto** aprendido con un codificador

al estilo VQ-VAE. El árbol de búsqueda alterna nodos de decisión y nodos de azar, y en los nodos de azar se ramifica por cada código posible.

En 2048 y en backgammon, juegos con azar, alcanza o supera el estado del arte y supera claramente a MuZero. En Go, que no tiene azar, rinde igual que MuZero ESTABLECIDO. Esta última observación es justamente la que pedía el recuadro anterior: **la ventaja aparece donde hay azar y desaparece donde no lo hay**. Es la firma de la pieza 4.

MoP-JEPA

> Fuente. *«MoP-JEPA: Hard-Assigned Predictor Mixtures for Stochastic JEPA World Models»* de Z. Song et al. (2026). [arXiv:2607.05238]

MoP-JEPA (Song et al., 2026) lleva el mismo argumento a los modelos del mundo tipo JEPA (capítulo VIII). Parte de un diagnóstico: con un solo predictor entrenado por regresión, en una bifurcación el predictor óptimo entrega la media condicional de los estados siguientes posibles, que no corresponde a ninguno. Los autores llaman **colapso** a este fenómeno (volveremos sobre esta palabra en el capítulo VII). La solución es tener K cabezas predictoras y asignar cada ejemplo de entrenamiento **solo a la cabeza que mejor lo predice** (asignación dura), con un enrutador que mira solo el contexto. Las cabezas terminan especializando, una por cada modo del futuro (Figura 13).

En laberintos de OGBench, planificar sobre los futuros que proponen las cabezas alcanza una tasa de éxito cercana a 0,85, frente a 0,02–0,09 de las variantes con una sola salida (según el resumen del paper) ESTABLECIDO.

> Para pensar *¿Hace falta que el **estado** sea discreto para elegir entre futuros? En Stochastic MuZero y en MoP-JEPA, ¿qué es exactamente lo discreto?*

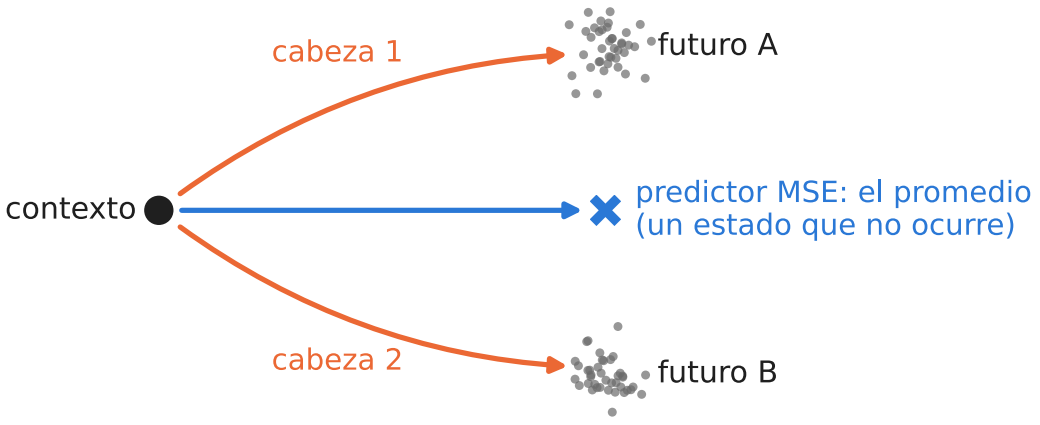


Figura 13. En una bifurcación, un predictor entrenado con MSE apunta al promedio de los futuros, que no es un estado real. Con varias cabezas y asignación dura, cada cabeza se queda con un futuro.

Qué se discretiza	La elección entre futuros (un código de azar, o qué cabeza usar). El estado sigue siendo continuo.
Qué mejora	Planificación en entornos estocásticos ESTABLECIDO sin efecto donde no hay azar (Go).
Qué empeora o cuesta	Hay que fijar cuántos modos (K); el número de ramas crece con cada paso de azar.
Pieza del paquete	4, de forma casi pura.

IV.5 Robustez

> Fuente. *«Discrete Representations Strengthen Vision Transformer Robustness»* de C. Mao et al. (2022). [arXiv:2111.10493]

Mao et al. (2022) agregan a la entrada de un *vision transformer* (ViT), junto a los parches de píxeles habituales, **tokens discretos** producidos por un codificador VQ. La robustez mejora hasta en un 12 % en siete benchmarks de ImageNet con imágenes alteradas (ruido, desenfoque, estilos artísticos, objetos fuera de contexto), sin perder precisión en las imágenes limpias ESTABLECIDO. La explicación que proponen es que los tokens discretos capturan la **forma** global y son insensibles a cambios pequeños de textura, porque un cambio pequeño no alcanza a mover un parche a otro código. Es la pieza 5, el redondeo, usada para ganar invarianza SUGERIDO.

IV.6 Dónde gana lo continuo

La evidencia anterior podría dejar la impresión de que discretizar siempre ayuda. Dos líneas de trabajo muestran lo contrario, y son tan informativas como las anteriores.

Generación de imágenes sin cuantización

> **Idea base** *Tokenizadores y pérdida de difusión*
>
> Los generadores de imágenes **autorregresivos** (como los modelos de lenguaje, pero con imágenes) trabajan sobre una secuencia de **tokens**: un **tokenizador** comprime la imagen en una grilla de latentes, y un transformer aprende a predecir cada token a partir de los anteriores. Si los tokens son discretos (VQ), la predicción es una clasificación con entropía cruzada. Si son continuos, hace falta otra forma de predecir una distribución. Una **pérdida de difusión** entrena una red pequeña para quitarle ruido a un token, condicionada en lo que dice el transformer. Al generar, se parte de ruido puro y se lo va limpiando hasta obtener un token: así se muestrea de una distribución continua que puede tener varios modos.

MAR (Li et al., 2024) compara directamente ambas opciones sobre ImageNet (Figura 14). Hay dos resultados:

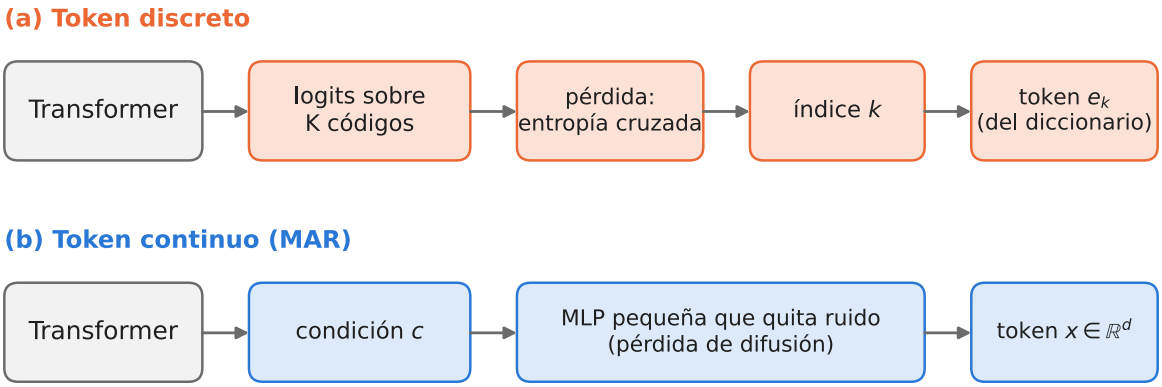


Figura 14. (a) El esquema clásico: el transformer clasifica cuál código viene, y el token es un vector del diccionario. (b) MAR: el transformer produce una condición, y una red pequeña de difusión genera un token continuo a partir de ella.

- **El tokenizador discreto pierde información.** El tokenizador VQ reconstruye peor que uno continuo de la misma resolución (rFID 5,87 frente a 1,43), y la generación lo arrastra: FID 7,82 frente a 3,50 ESTABLECIDO. Es el costo de la pieza 1.
- **La discreción no es necesaria para modelar.** Con el **mismo** tokenizador VQ, predecir con pérdida de difusión da mejor FID (7,82) que predecir con entropía cruzada (8,79) ESTABLECIDO. Lo que el modelo autorregresivo necesitaba era una pérdida que represente una distribución con varios modos (pieza 4), y la entropía cruzada sobre códigos era solo una forma de conseguirla.

Acciones latentes en video real

> Fuente. *«Learning Latent Action World Models In The Wild»* de Q. Garrido et al. (2026). [arXiv:2601.05230]

Un **modelo de acciones latentes** aprende un modelo del mundo a partir de videos que no traen las acciones anotadas. Para eso, un módulo mira dos cuadros consecutivos e infiere una «acción latente» que explica el cambio, y otro módulo predice el cuadro siguiente a partir del actual y de esa acción. Hay una trampa: si la acción latente puede llevar mucha información, el modelo simplemente copia el cuadro siguiente dentro de ella. Por eso hay que **limitar su capacidad**. Genie (Bruce et al., 2024) lo hizo con un diccionario VQ de pocos códigos.

Garrido et al. (2026) entrenan estos modelos con video «en la naturaleza» (videos de internet, sin una única encarnación ni un conjunto fijo de acciones). Encuentran que las acciones VQ **no logran capturar** la complejidad de las acciones reales, mientras que acciones **continuas** limitadas con esparsidad o con ruido sí lo logran ESTABLECIDO. Lo que importaba era limitar la información (pieza 1), y la discretización era una forma demasiado tosca de hacerlo cuando el espacio de acciones es rico.

IV.7 Resumen del capítulo

Caso	Qué se discretiza	Resultado	Pieza
DreamerV2	estado estocástico	mejora en 42/55 juegos	sin separar
Meyer et al.	estado (multi-one-hot)	mejora con poca capacidad y en cambios	2 1
Meyer et al.	mismo código, como vector	mucho peor que *one-hot*	2
DCWM	estado (FSQ) + clasificación	mejora; vectores > *one-hot*	4 3 2
Stochastic MuZero	resultado del azar	mejora con azar, igual sin azar	4
MoP-JEPA	elección de cabeza	0,85 frente a 0,02–0,09	4
Mao et al.	tokens extra de entrada	hasta +12 % de robustez	5
MAR	tokens de imagen	**continuo gana**	1 4
Garrido et al.	acciones latentes	**continuo gana**	1

Tabla 1. Evidencia del capítulo, atribuida a piezas del paquete.

Qué nos llevamos

- La creencia de que «lo discreto es mejor» nació de DreamerV2, cuya ablación no separa causas. Los propios autores no saben por qué funciona.
- Los experimentos que sí separan causas muestran efectos **condicionales**: la ventaja aparece con poca capacidad (Meyer), con azar (Stochastic MuZero, MoP-JEPA) o con geometría adecuada (*one-hot* en laberintos, vectores en control).
- Mismo código, distinta codificación, resultados muy distintos: lo que importa no es solo que sea discreto.
- Lo continuo gana cuando el espacio es rico (acciones reales, texturas de imágenes) y cuando se dispone de otra forma de modelar varios modos (difusión).
- Casi todos los éxitos discretizan **una** cosa (la elección, la pérdida, el estado estocástico) y conservan una parte continua. El diseño con mejor respaldo es híbrido.

Capítulo V

Seguir el error

En una frase *En un espacio continuo todo punto es un estado posible, así que un error se ve igual que un estado legítimo; en un espacio discreto, los estados válidos son pocos y están separados, y eso convierte el error en algo que se puede detectar, borrar, ubicar en el tiempo y contar.*

Se repite a menudo que la discretización «permite seguir el error» (*track errors*). La frase es correcta, pero suele quedarse en una intuición. Este capítulo la desarma en cuatro capacidades distintas: **detectar** que hubo un error, **corregirlo**, **localizar** en qué paso ocurrió y **contar** cuántos hubo. Para cada una veremos qué la hace posible, cuánto rinde y cuándo deja de funcionar. Es el desarrollo de la pieza 5 (redondeo), con apoyo de la 6 (identidad exacta).

V.1 El problema: en continuo, el error es invisible

Pensemos en un modelo del mundo que predice la posición de un robot sobre una línea. La posición real es 1,00 y el modelo predice 1,27. ¿Puede el modelo, mirando solo su propia predicción, darse cuenta de que se equivocó?

No puede. En un espacio continuo, 1,27 es una posición tan válida como 1,00 (Figura 15, arriba). Nada en el valor delata el error. El error existe, pero es invisible desde dentro.

Ahora supongamos que el robot solo puede estar en posiciones enteras: 0, 1, 2, 3 o 4. Entonces 1,27 **no es un estado posible** (Figura 15, abajo). El solo hecho de caer entre dos estados válidos dice «aquí hubo un error». Además, el estado válido más cercano, 1, es una buena apuesta de cuál debía ser.

Es lo mismo que pasa al leer. Si alguien escribe «gaot», sabemos que hubo un error, porque «gaot» no es una palabra, y podemos adivinar que quiso decir «gato». El idioma tiene muchas menos palabras que combinaciones de letras, y esa **redundancia** es la que permite detectar y corregir. Un espacio continuo no tiene redundancia de este tipo: cada combinación de valores «es una palabra».

> **Idea base** *Códigos correctores de errores*
>
> Shannon (1948) y Hamming (1950) fundaron la teoría de cómo transmitir información sin errores por un canal con ruido. La idea central es usar solo un subconjunto de los mensajes posibles (las

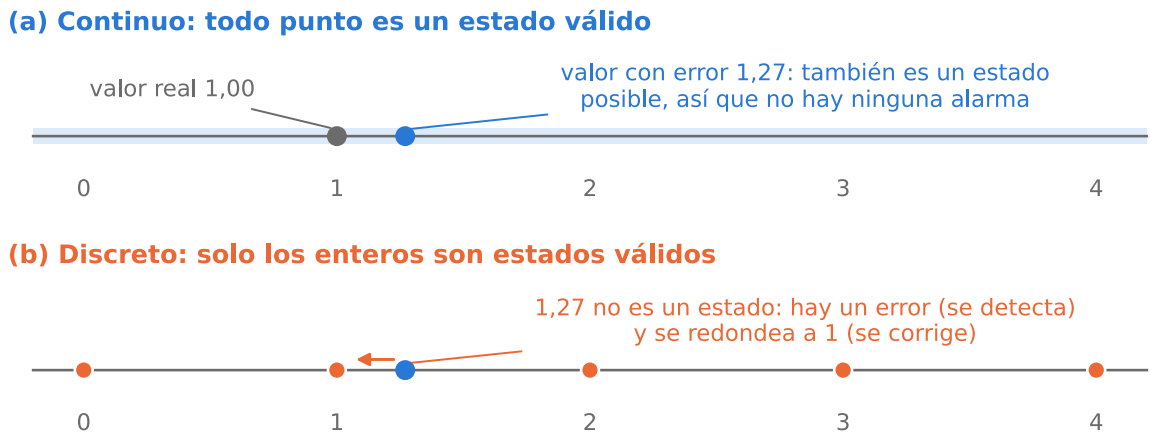


Figura 15. (a) En un espacio continuo, el valor con error es indistinguible de un estado legítimo. (b) Si solo los enteros son estados válidos, caer entre ellos revela el error (detección), y redondear al más cercano lo elimina (corrección).

palabras de código) y elegirlas **lejos** unas de otras. Si dos palabras de código válidas difieren en al menos d posiciones (su **distancia mínima**), entonces:

- se pueden **detectar** hasta $d-1$ errores, porque con menos de d cambios no se alcanza otra palabra válida;
- se pueden **corregir** hasta $\lfloor (d-1)/2 \rfloor$ errores, porque con esos cambios la palabra original sigue siendo la más cercana.

Hoy, toda la memoria de un computador, los discos y las comunicaciones móviles usan este principio.

V.2 Detectar y corregir: la geometría de los márgenes

La misma idea funciona en un espacio latente continuo donde los códigos válidos son unos pocos puntos $e_1, \ldots, e_K$, como el diccionario de un VQ-VAE. Llamemos d a la distancia entre dos códigos vecinos. Supongamos que el modelo debía producir el código A, pero su salida tiene ruido. La Figura 16 muestra qué puede pasar.

Si el sistema acepta un punto cuando está a menos de r de algún código, y lo reemplaza por ese código, aparecen tres zonas:

- **Se corrige:** el error mide menos que r. El punto vuelve a A y el error desaparece por completo.
- **Se detecta:** el punto queda a más de r de todos los códigos. El sistema no sabe cuál era, pero **sabe que algo anda mal** y puede pedir otra observación, marcar la predicción como poco confiable o detenerse.
- **Error silencioso:** el error es tan grande (más que $d-r$) que el punto cae cerca de **otro** código. El sistema lo corrige hacia el código equivocado y no se entera.

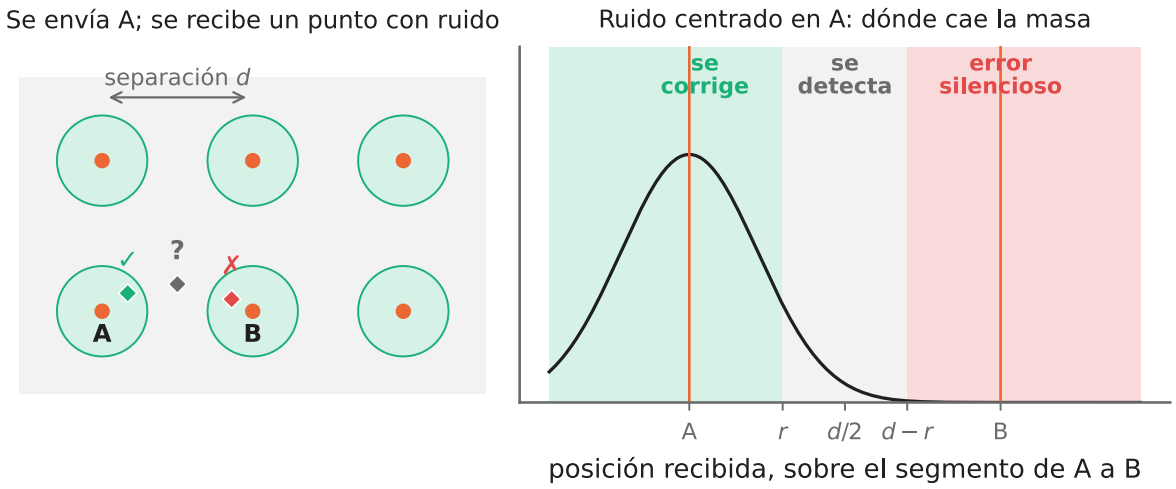


Figura 16. Izquierda: códigos válidos (naranjos) separados por d, con un disco de radio r alrededor de cada uno. Se envía A y se recibe un punto con ruido. Si cae en el disco de A, se corrige (✓). Si cae lejos de todo código, se detecta el error (?). Si cae en el disco de otro código, se produce un error silencioso (✗). Derecha: el mismo análisis sobre el segmento de A a B, con una distribución de ruido centrada en A.

El umbral r reparte el espacio entre las tres zonas. Con $r = d/2$ no hay zona de detección: todo se corrige, bien o mal, que es lo que hace un VQ-VAE al tomar siempre el vecino más cercano. Con r más pequeño se corrige menos, pero se detecta más y se cometen menos errores silenciosos.

Hay un segundo compromiso, más profundo. Para que los errores frecuentes caigan en la zona de corrección, los códigos tienen que estar lejos entre sí. Pero en una región de tamaño fijo, **cuanto más separados estén los códigos, menos caben**. En n dimensiones, con separación d en una región de lado A, caben del orden de $(A/d)^n$ códigos. Más margen significa más robustez y menos capacidad. La pieza 5 (redondeo) y la pieza 1 (capacidad) son dos caras del mismo diseño.

> **Para pensar** *En un VQ-VAE, ¿qué pasa con la capacidad de corregir errores si dos códigos del diccionario terminan muy cerca uno del otro?*

V.3 Corregir en cadena: errores que no se acumulan

La corrección se vuelve decisiva cuando la salida de un paso es la entrada del siguiente, como en los rollouts de un modelo del mundo (Figura 5). Veamos un modelo mínimo que permite calcular todo.

Supongamos que el estado verdadero avanza de forma determinista y que, en cada paso, el modelo agrega un error aleatorio ε_t con distribución normal de desviación σ. Midamos todo en unidades de la separación d entre códigos.

En continuo, los errores se suman:

$$e_T = \varepsilon_1 + \varepsilon_2 + \ldots + \varepsilon_T, \quad \mathrm{Var}(e_T) = T\sigma^2.$$

El error típico crece como $\sigma\sqrt{T}$, sin límite. Y esto es el caso optimista. Si la dinámica amplifica las diferencias (si dos estados cercanos se separan con el tiempo, con un factor $L > 1$ por paso), el error crece como L^T, es decir, exponencialmente.

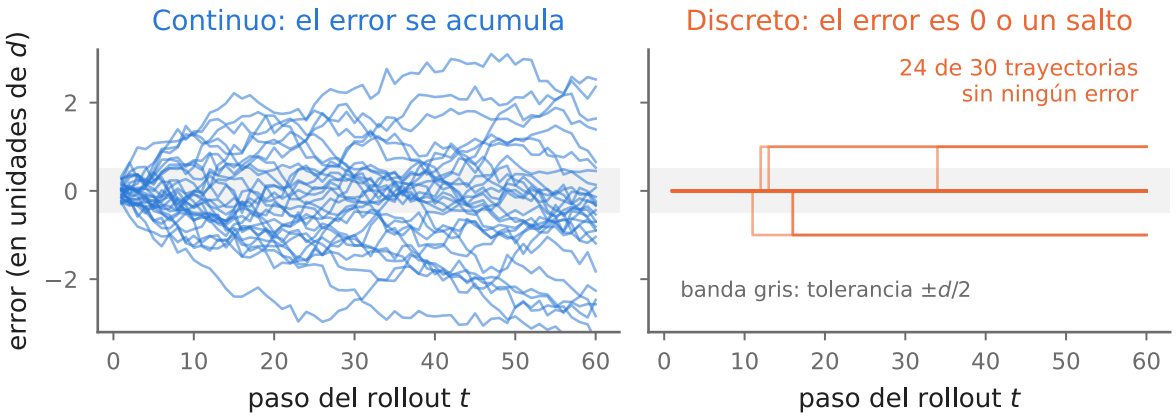


Figura 17. Simulación con 30 rollouts de 60 pasos y ruido $\sigma = 0,18d$ por paso (semilla fija; el código está en `figures/generar.py`). En continuo, el error de todos los rollouts se dispersa como $\sigma\sqrt{t}$. En discreto, 24 de los 30 rollouts terminan sin **ningún** error, y los demás tienen uno o dos saltos.

En discreto, después de cada paso se redondea al código más cercano. Si el error acumulado es cero y el ruido de este paso mide menos que $d/2$, el redondeo lo borra y el error **sigue siendo exactamente cero**. Solo cuando un ruido supera la mitad de la separación el estado salta a un código vecino. La probabilidad de que eso pase en un paso es

$$p = P(|\varepsilon| > d/2) = 2\overline{\Phi}\left(\frac{d}{2\sigma}\right),$$

donde $\overline{\Phi}$ es la cola de la distribución normal. Así, el error discreto no es una suma de ruidos pequeños, sino una sucesión de ceros interrumpida, de vez en cuando, por un salto completo. La Figura 17 lo muestra con una simulación.

¿Cuánto mejor es? Una forma limpia de compararlos es mirar la **variabilidad que se agrega en cada paso**. En continuo es σ^2. En discreto es la varianza de un salto, que para ruido pequeño es aproximadamente $p \cdot d^2$. La tabla muestra ambas para distintos niveles de ruido (en unidades de d^2):

Ruido σ	Continuo: σ^2	Discreto: varianza del salto	Razón
$0,10d$	$0,0100$	$0,00000057$	$\approx 1/17000$
$0,15d$	$0,0225$	$0,00086$	$\approx 1/26$
$0,20d$	$0,0400$	$0,0124$	$\approx 1/3$
$0,25d$	$0,0625$	$0,0455$	$\approx 0,7$
$0,35d$	$0,1225$	$0,153$	$\approx 1,25$

La tabla tiene dos lecturas, y las dos importan:

1. **Cuando el ruido por paso es pequeño frente al margen, el redondeo es enormemente eficaz.** Con $\sigma = 0,1d$, el error discreto crece unas 17 000 veces más lento que el continuo. Es un efecto de **umbral**: la probabilidad de salto cae como la cola de una gaussiana, mucho más rápido que σ^2.
2. **Cuando el ruido es comparable al margen, el redondeo empeora las cosas.** Desde $\sigma \approx 0,29d$ (el cruce exacto), un salto discreto cuesta más que el ruido que reemplaza. Redondear convierte errores pequeños en saltos de un código completo.

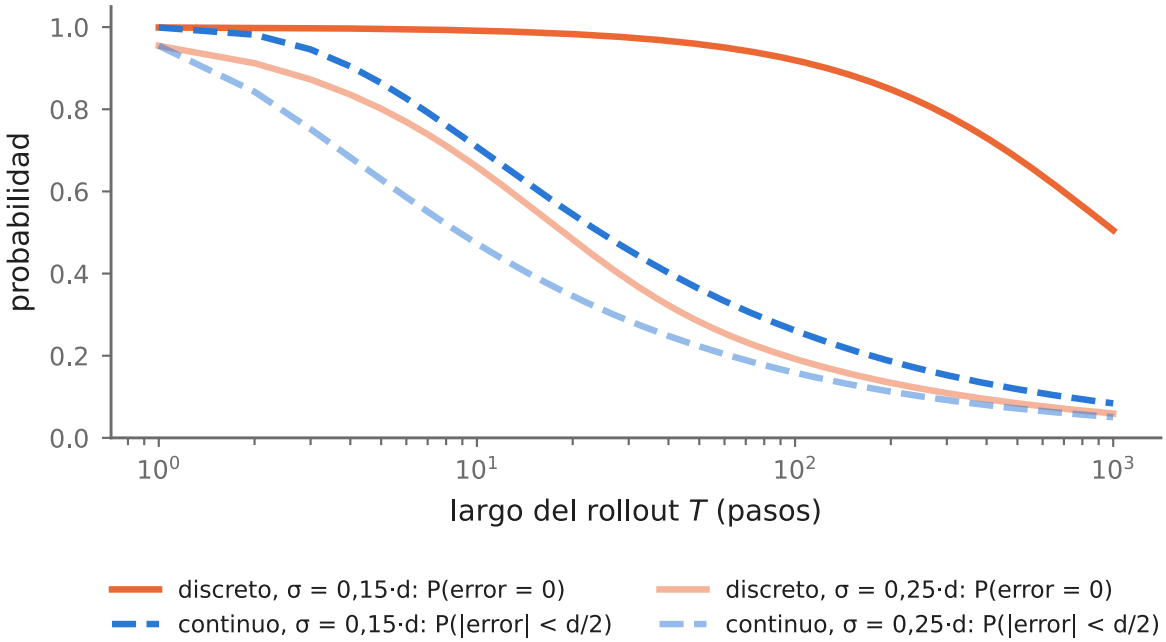


Figura 18. Probabilidad de que el error después de T pasos sea menor que $d/2$, calculada de forma exacta para el modelo del texto. En discreto, eso equivale a que el error sea exactamente cero. Con $\sigma = 0,15d$, el rollout discreto sigue siendo exacto con probabilidad cercana a 1 durante cientos de pasos; el continuo la pierde en decenas. Con $\sigma = 0,25d$, la ventaja casi desaparece.

La Figura 18 muestra la consecuencia sobre rollouts largos: la probabilidad de que el estado siga siendo correcto después de T pasos.

Este modelo es deliberadamente simple, pero explica tres observaciones del capítulo IV:

- En Meyer et al., los modelos continuos fallan justamente en las transiciones **raras**, donde el modelo es menos preciso y el error se arrastra por los 30 pasos del rollout.
- La ventaja discreta **desaparece con modelos grandes**: un modelo grande tiene σ pequeño, y ambos rollouts se vuelven precisos.
- La cuarta hipótesis de DreamerV2, la de los aspectos «no suaves» del mundo: donde el mundo tiene saltos (entrar a una habitación), un estado continuo tiende a quedar «a medio camino», y el redondeo lo empuja a un lado o al otro.

> **Idea base** *Atractores de Hopfield*
>
> Una **red de Hopfield** (Hopfield, 1982) es una red neuronal cuya dinámica tiene unos pocos estados estables, los **atractores**, que funcionan como memorias. Si se la inicia con una versión ruidosa o incompleta de una memoria, la dinámica la lleva al atractor más cercano: la memoria completa. Es el redondeo implementado por una dinámica continua, y muestra que la corrección de errores no exige que todo el sistema sea discreto, sino que tenga **estados preferidos separados entre sí**.

Hay un detalle del modelo que conviene notar: con redondeo, la amplificación L^T de la dinámica deja de importar mientras el error sea cero, porque un error nulo amplificado sigue siendo nulo. El

redondeo **corta la cadena de amplificación** en cada paso. Solo cuando ocurre un salto la dinámica vuelve a jugar en contra.

Esta discusión es la primera mitad de la respuesta a la nota del profesor (*«accumulated errors =?= collapse»*). Aquí vimos cómo se acumula el error y cómo el redondeo lo detiene. En el capítulo VII veremos que el colapso aparece en el mismo tipo de bucle (una salida que vuelve a ser entrada u objetivo), pero en el régimen opuesto: cuando el bucle **contrae** demasiado.

V.4 Localizar y contar: el error como evento

Las secciones anteriores trataban de **borrar** el error. Pero «seguir el error» también significa poder **decir dónde ocurrió**. Aquí la pieza que trabaja es la [6]: en un espacio discreto, dos estados son iguales o no lo son.

Consideremos una cadena de pasos, como un programa que se ejecuta instrucción por instrucción o un rollout que avanza. Si cada estado intermedio es un código, se puede comparar **exactamente** con el estado correcto (Figura 19, arriba). El resultado de cada comparación es un sí o un no, y aparece algo que en continuo no existe: **el primer paso en que la cadena se equivocó**. El error deja de ser una cantidad difusa y pasa a ser un **evento**, con fecha y lugar.

En una cadena continua (Figura 19, abajo) todos los pasos tienen **algo** de error, y este crece de a poco. Para decir «aquí hubo un error» hay que elegir un umbral, y la respuesta cambia con el umbral. No hay un paso culpable, sino una responsabilidad repartida entre todos.

Tener eventos en vez de cantidades difusas habilita cuatro cosas concretas:

- **Depurar.** Si una ejecución falla, se puede buscar el primer paso incorrecto y estudiar por qué. Es lo que hace un programador, y es lo que permite que DreamCoder (Ellis et al., 2021) verifique un programa contra los ejemplos de forma exacta: el programa es correcto o no lo es.
- **Asignar crédito.** En el entrenamiento, se puede concentrar la corrección en el paso que falló en vez de repartirla entre todos.

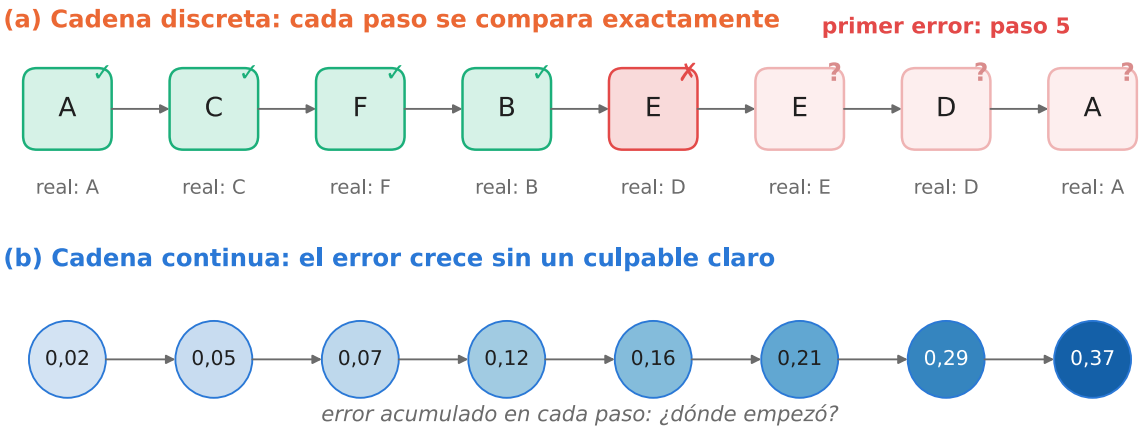


Figura 19. (a) En una cadena discreta, cada paso se compara exactamente con el correcto: el primer error ocurre en el paso 5, y los siguientes lo heredan. (b) En una cadena continua, todos los pasos tienen algo de error; para hablar de «un error» hay que elegir una tolerancia arbitraria, y no hay un paso culpable.

- **Contar y predecir.** Si cada paso falla con probabilidad p_t de forma independiente, la probabilidad de que una cadena de T pasos salga perfecta es

$$P(\text{cadena correcta}) = \prod_{t=1}^{T}(1 - p_t) \approx e^{-\sum_t p_t}.$$

  Eso permite **predecir** cómo cae el rendimiento al alargar las cadenas, por ejemplo al pedirle a NLI programas más largos que los de entrenamiento.
- **Medir.** Meyer et al. pudieron comparar distribuciones de estados del modelo y del entorno porque los estados de un laberinto son contables. En un espacio continuo, la misma comparación exige discretizar **para medir**.

Hay una quinta capacidad, más especulativa. En un VQ-VAE, la distancia entre la salida del codificador y su código más cercano, $\|z_e - e_k\|$, es una **alarma gratuita**: si es grande, la entrada no se parece a nada de lo que el modelo conoce. En principio, eso sirve para detectar situaciones nuevas (conjetura).

V.5 El precio y los límites

Seguir el error no es gratis, y es importante tener claros sus límites:

- **Los errores silenciosos son catastróficos.** Cuando un error supera el margen, el sistema salta a otro código **con total confianza**. En continuo, un error grande al menos se ve grande. En discreto, un error grande puede verse como un estado perfectamente normal.
- **El error de cuantización es invisible.** Si el estado verdadero está **entre** dos códigos, el sistema siempre se equivoca hasta en $d/2$ y nunca lo detecta, porque la alarma solo suena para desvíos respecto de los códigos, no respecto de la verdad. Es el costo de la pieza ❶ que vimos en MAR y en Garrido et al.
- **Hace falta margen, y el margen cuesta capacidad.** Más separación entre códigos significa menos códigos en el mismo espacio.
- **Solo funciona si el ruido por paso es menor que el margen.** El cruce de la tabla ($\sigma \approx 0,29d$) es un límite duro: con un modelo impreciso, redondear empeora las cosas.

V.6 ¿Es exclusivo de lo discreto?

Volvamos a lo que realmente hizo falta en cada caso:

1. que los estados válidos ocupen **solo una parte** del espacio, con margen entre ellos (redundancia);
2. una forma de **volver** a esa parte cuando uno se sale (proyección).

Lo discreto es la forma más simple de tener las dos cosas, pero no la única. Hay sistemas continuos que también las tienen:

- **Restricciones conocidas.** Si el estado debe cumplir una ley (conservar la energía, que una rotación tenga norma uno), cada predicción se puede proyectar de vuelta a la superficie de estados que la cumplen. Los errores perpendiculares a esa superficie se borran.
- **Atractores continuos.** Las redes de Hopfield son dinámicas continuas con estados preferidos separados.

- **Filtros que estiman su propio error.** El filtro de Kalman (Kalman, 1960) no borra el error, pero **lleva la cuenta** de cuán incierto está su estado y combina predicción y observación en consecuencia. Es «seguir el error» en el sentido de medirlo, a cambio de necesitar un modelo del ruido.

Lo que sí es más propio de lo discreto es la identidad exacta (pieza 6): poder decir «este paso es correcto» sin elegir una tolerancia. Esto anticipa la respuesta del capítulo VI a la pregunta A1. El principio no parece ser «discretizar», sino **tener estados preferidos con margen y una forma de volver a ellos**, y la discretización es su versión más barata y extrema (conjetura).

Cómo explicarlo en dos minutos

En un espacio continuo, estar un poco equivocado se ve igual que estar en otro estado legítimo, así que el error es invisible y se acumula en silencio. Al discretizar, solo unos pocos estados separados son válidos. Esto tiene tres consecuencias:

- caer entre ellos **delata** el error;
- volver al más cercano lo **borra**, si era pequeño;
- como dos estados son iguales o distintos, se puede **señalar el paso** en que la cadena se equivocó y **contar** los errores.

El precio es que un error grande se convierte en un salto a otro estado, con total confianza. Por eso el redondeo solo conviene cuando el ruido por paso es claramente menor que la mitad de la separación entre estados.

Qué nos llevamos

- «Seguir el error» son cuatro capacidades: detectar, corregir, localizar y contar. Todas dependen de que los estados válidos sean pocos y estén separados.
- En una cadena, el redondeo transforma una acumulación de ruido ($\mathrm{Var} = T\sigma^2$) en saltos raros (probabilidad $2\overline{\Phi}(d/(2\sigma))$ por paso). Con ruido pequeño frente al margen la mejora es enorme; desde $\sigma \approx 0,29d$ el redondeo empeora las cosas (establecido) (resultado matemático del modelo simple).
- Con estados discretos el error se vuelve un evento con fecha: permite depurar, asignar crédito, predecir el rendimiento en cadenas más largas y medir.
- Los costos son los errores silenciosos, el error de cuantización invisible y el compromiso entre margen y capacidad.
- Lo esencial no parece ser lo discreto, sino tener estados preferidos con margen y una proyección hacia ellos (conjetura).

Capítulo VI

¿Es la discretización?

En una frase *Cada pieza del paquete predice una «firma» distinta en los experimentos; casi todos los beneficios documentados se pueden reproducir sin discretizar, atacando la pieza responsable, y lo único que no tiene sustituto continuo claro es la identidad exacta.*

Este capítulo responde a P3 (¿qué experimento comprueba o desmiente que la discretización es la causa?) y a la pregunta abierta A1 (¿hay un principio más allá de la discreción?). La estrategia es la misma de todo el texto. Si «discretizar» es un paquete, la pregunta «¿ayuda discretizar?» se reemplaza por seis preguntas más pequeñas, una por pieza, y cada una se puede poner a prueba por separado.

VI.1 Desarmar el paquete

La firma de cada pieza

Cada pieza, si fuera la responsable de una mejora, dejaría una huella distinta en los experimentos. La tabla resume qué habría que observar.

Pieza	Si fuera la responsable, veríamos que…	Experimento que lo distingue
❶ Capacidad	la ventaja desaparece al darle al continuo el mismo número de bits (con ruido o esparsidad)	discreto frente a continuo con capacidad igualada
❷ Geometría	el mismo código rinde distinto según cómo se entregue	*one-hot* frente a vector, con la misma información
❸ Pérdida	clasificar ayuda aunque el estado siga siendo continuo	MSE frente a CE sobre el mismo latente
❹ Modos	la ventaja aparece solo con azar y desaparece en entornos deterministas	mismo método en un entorno con azar y en otro sin azar
❺ Redondeo	la ventaja crece con el largo del rollout y depende del cociente ruido/margen	error a 1 paso frente a error a 30 pasos
❻ Identidad	la ventaja aparece al buscar, reutilizar o componer, y no en predicción de un paso	tareas con y sin búsqueda o composición

Tabla 2. La «firma» de cada pieza: qué observación la delataría.

Los confusores

Un **confusor** es algo que cambia junto con la variable que nos interesa y que podría explicar el efecto por sí solo. En los experimentos de discretización aparecen siempre los mismos:

- **La pérdida.** Pasar a códigos suele traer consigo pasar de MSE a entropía cruzada. Farebrother et al. (2024) muestran que predecir valores por clasificación (con HL-Gauss) en vez de regresión mejora el aprendizaje por refuerzo profundo en muchos dominios, **sin discretizar ningún estado**. DreamerV3 hace lo mismo con sus objetivos *two-hot*.
- **La capacidad.** Un diccionario de K códigos lleva $\log_2 K$ bits; un vector continuo del mismo tamaño puede llevar muchísimos más. Si no se igualan, se compara un modelo con cuello de botella contra uno sin él.
- **El tamaño del modelo.** En Meyer et al., el efecto depende de la capacidad del modelo de transición y desaparece con modelos grandes.
- **La arquitectura que acompaña a lo discreto.** En NLI, los tokens discretos vienen con largo variable y con un intérprete secuencial; en LPN, el latente continuo tiene largo fijo.
- **El estimador del gradiente.** El *rotation trick* mejora mucho un VQ sin cambiar la representación, solo cambiando cómo pasa el gradiente.
- **Los hiperparámetros.** Discrete-JEPA (Baek et al., 2025) compara su versión discreta con I-JEPA usando dimensiones de token distintas (96 frente a 768) y tasas de aprendizaje distintas.

¿Qué experimentos ya existen?

La Figura 20 cruza las piezas con los experimentos que vimos y marca cuáles aíslan cada pieza y cuáles solo la tocan.

Tres observaciones:

1. Las piezas 2, 3 y 4 están bien cubiertas. Hay experimentos limpios que muestran que la geometría, la pérdida y la capacidad de elegir entre modos importan **cada una por separado**.
2. La pieza 1 (capacidad) está cubierta desde el lado continuo. Garrido et al. muestran que un continuo con capacidad limitada supera al VQ, y la tabla 14 de I-JEPA muestra que estrechar el predictor (de 1024 a 384 canales) **mejora** el resultado (de 68,4 a 70,7) ESTABLECIDO. Es un cuello de botella que ayuda sin discretizar nada.
3. La pieza 5 solo tiene evidencia parcial, y la 6 ninguna. Discrete-JEPA es sugerente: I-JEPA gana en los primeros pasos y Discrete-JEPA gana a largo plazo (en el paso 1000, el LPIPS es 0,0242 frente a 0,1554). Es justo la firma del redondeo, pero con hiperparámetros distintos SUGERIDO.

VI.2 Un experimento para desarmar el paquete (P3)

La matriz muestra dónde faltan experimentos. Esta sección propone uno que llena esos huecos y que se podría montar con los modelos del curso. La idea es un **diseño factorial**: en vez de cambiar todo a la vez, se cruzan todas las combinaciones de unos pocos factores (Figura 21).

Los factores son:

- **Representación**: latente continuo o discreto.

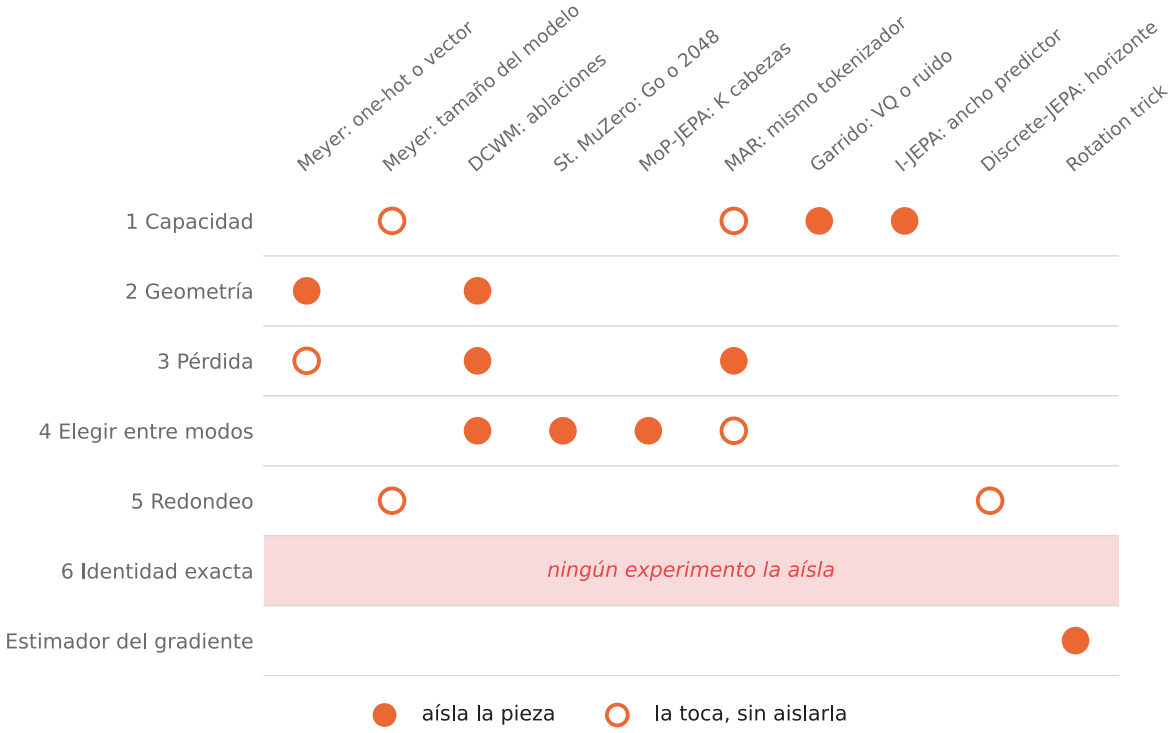


Figura 20. Qué experimentos publicados aíslan cada pieza del paquete. Un círculo lleno significa que el experimento cambia esa pieza y nada más (o casi nada más). Un círculo vacío significa que la cambia junto con otras. La fila de la identidad exacta está vacía.

- **Capacidad**: libre, o igualada. Para igualarla, al latente continuo se le suma ruido gaussiano calibrado para que lleve aproximadamente $\log_2 K$ bits por código, como el discreto.

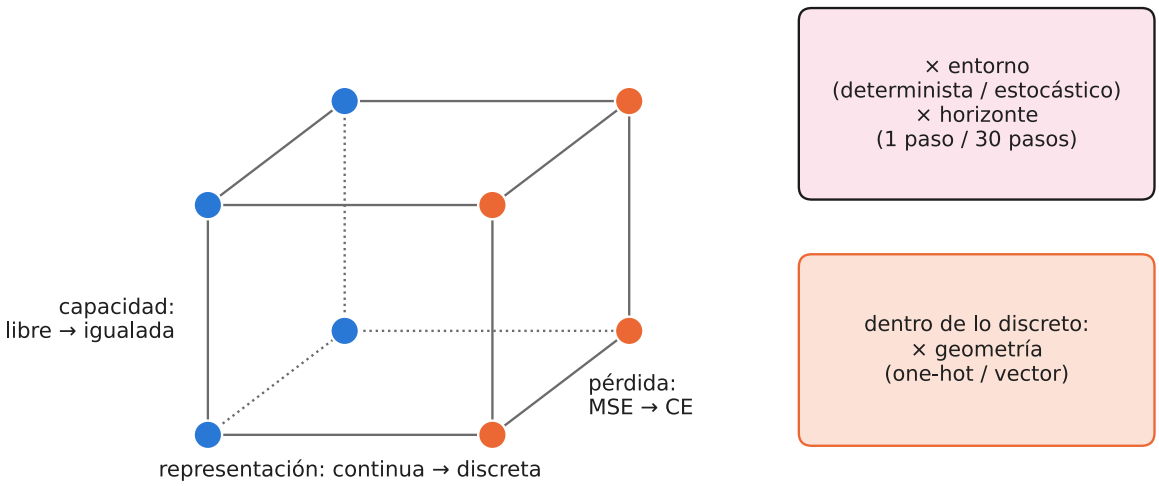


Figura 21. El diseño propuesto. Cada vértice del cubo es una variante del modelo; azul es continuo y naranjo es discreto. Cada variante se evalúa en entornos con y sin azar y con horizontes corto y largo. Dentro de lo discreto, se cruza además la geometría.

- **Pérdida**: MSE o entropía cruzada. Para el continuo, la entropía cruzada se aplica discretizando **solo el objetivo** (HL-Gauss), sin tocar el estado.
- **Entorno**: determinista o estocástico.
- **Horizonte**: error a un paso y a treinta pasos.
- **Geometría** (solo para el discreto): *one-hot* o vector.

Lo valioso del diseño es que cada hipótesis predice un patrón distinto:

- Si la causa es la **capacidad**, la diferencia entre continuo y discreto desaparece en las variantes con capacidad igualada.
- Si es la **pérdida**, desaparece cuando ambos usan entropía cruzada.
- Si es **elegir entre modos**, aparece solo en el entorno estocástico.
- Si es el **redondeo**, es pequeña a un paso y grande a treinta.
- Si es la **geometría**, la variante discreta con vectores pierde parte de la ventaja.
- Si después de controlar todo lo anterior queda una diferencia, hay algo propio de la discreción que nuestras piezas no capturan.

Aplicado a LPN y NLI. La pregunta del inicio (¿por qué NLI generaliza mejor que LPN?) se presta para este diseño, con cuatro variantes concretas:

1. Un LPN con **largo variable**: una secuencia de latentes continuos leída por el intérprete de NLI. Separa la discreción del largo variable.
2. Un NLI con **largo fijo**. Es la misma separación, en sentido contrario.
3. Un NLI con **tokens continuos** y ruido calibrado. Separa la discreción de la capacidad.
4. Un NLI que entrega sus tokens como **vectores** en vez de *one-hot*. Mide la geometría.

La métrica sería la generalización composicional: el éxito al componer 2, 3 y 4 o más primitivas, que es donde LPN fallaba. Una predicción concreta, que el experimento podría desmentir: si la ventaja de NLI viene sobre todo del largo variable y del intérprete secuencial, las variantes 1 y 3 deberían recuperar buena parte de ella; si viene de la identidad exacta (pieza 6), no deberían recuperarla CONJETURA.

> **Para pensar** *Si en este experimento la variante 3 (tokens continuos con ruido calibrado) rindiera igual que NLI, ¿qué concluirías sobre la pregunta del reporte de la semana 9?*

VI.3 ¿Hay un principio más allá de la discreción? (A1)

Qué se pudo reproducir sin discretizar

La forma más directa de responder A1 es revisar, beneficio por beneficio, si alguien lo obtuvo **sin** discretizar.

Beneficio	¿Sin discretizar?	Cómo
Elegir entre modos 4	sí	pérdida de difusión sobre tokens continuos (MAR); mezclas de gaussianas, aunque estas rinden peor en Meta-World (DCWM)
Limitar capacidad 1	sí	acciones continuas con ruido o esparsidad (Garrido); predictor angosto en I-JEPA

Beneficio	¿Sin discretizar?	Cómo
Clasificar [3]	sí	objetivo discretizado, estado continuo (HL-Gauss, *two-hot*)
Esparsidad [2]	en parte	la activación esparsa FTA recupera parte de la brecha: 574 frente a 382 y 674 (Meyer)
Estabilidad [5]	en parte	ortogonalidad de factores en JEPA-Anything (controla la amplificación del error al recomponer); atractores; proyección a restricciones
Identidad exacta [6]	no	no hay un sustituto continuo claro para enumerar, comparar exactamente o reutilizar

Tabla 3. Beneficios de la discretización y si se obtuvieron por otra vía.

La respuesta, entonces, es: **casi siempre sí**. Para cuatro de los seis beneficios hay una vía continua documentada; para uno hay vías parciales; y para uno no la hay.

El principio candidato

¿Qué tienen en común las vías continuas que funcionan? Todas imponen alguna de estas dos propiedades:

> **Idea base** *Invarianza y margen*
>
> Una representación es **invariante** a un cambio si ese cambio no la mueve. Por ejemplo, la representación de «gato» debería ser invariante a la iluminación. Tiene **margen** entre dos cosas que deben distinguirse si sus representaciones están lejos en comparación con el ruido. Una representación útil es invariante a lo que no importa para la tarea y tiene margen entre lo que sí importa. El aprendizaje contrastivo hace exactamente eso: junta las vistas de una misma imagen (invarianza) y separa las de imágenes distintas (margen).

La discretización es la forma **extrema** de imponer ambas cosas. Dentro de una celda, la invarianza es total: todos los puntos se vuelven el mismo código. Entre celdas, el margen es fijo e igual a la separación. Por eso proponemos el siguiente principio (conjetura):

Principio candidato (respuesta tentativa a A1)

Lo que mejora los resultados no es la discreción, sino una representación con **invarianza dentro de cada clase**, **margen entre clases** y **cortes que coinciden con la estructura del mundo**. La discretización es la manera más barata y extrema de conseguir las dos primeras. Ayuda cuando sus cortes coinciden con los del mundo y perjudica cuando no coinciden, porque entonces la invarianza borra diferencias que importaban.

La Figura 22 lo ilustra. Los mismos datos pueden representarse sin margen (el ruido cruza la frontera), con margen e invarianza en un espacio continuo (el ruido no alcanza a cruzar) o de forma discreta (el ruido, además, se borra al redondear). Las dos últimas resuelven la tarea; la discreta, además, permite seguir el error (capítulo V).

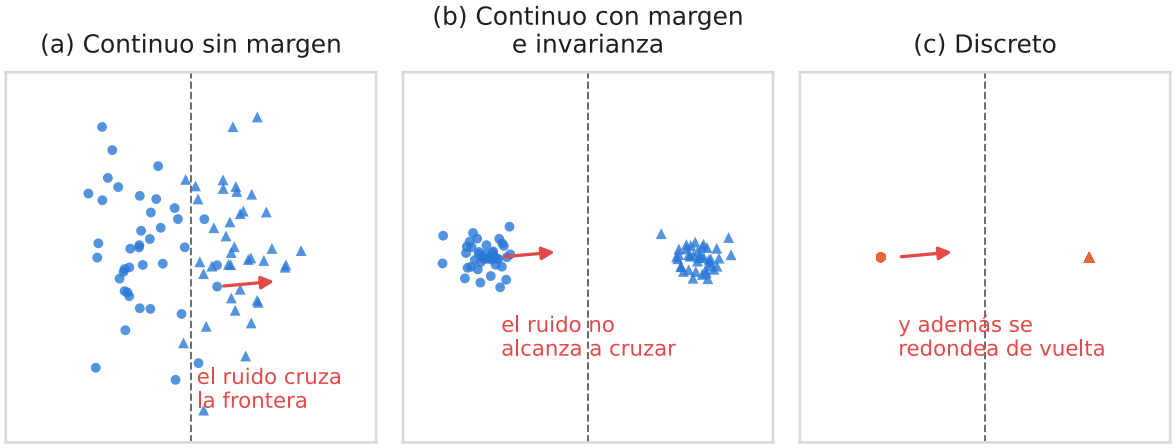


Figura 22. El principio en dos dimensiones. Dos clases (círculos y triángulos) separadas por una frontera (línea punteada). Lo que importa no es si la representación es discreta, sino si tiene margen frente al ruido; la discretización es el caso extremo.

El principio explica los casos del capítulo IV:

- En los laberintos (Meyer), los cortes naturales son categorías sin orden, y *one-hot* da margen máximo entre ellas.
- En control continuo (DCWM), los cortes tienen orden, y los vectores del diccionario conservan ese orden.
- En imágenes (MAR) y en acciones reales (Garrido), no hay cortes naturales gruesos: la invarianza de VQ borra detalle útil, y gana el continuo con capacidad limitada.

Además, conecta con un fenómeno del capítulo VII. Cuando una red se entrena con etiquetas por mucho tiempo, el descenso de gradiente lleva por sí solo las representaciones de cada clase a puntos casi idénticos, ubicados en los vértices de un símplex: el **colapso neuronal**. Es invarianza máxima y margen máximo, una **discretización que emerge** sin que nadie la imponga.

Lo que sí parece propio de lo discreto

La única fila sin sustituto continuo en la Tabla 3 es la identidad exacta. Poder decir «este estado es **el mismo** que aquel» permite tres cosas:

- **enumerar** hipótesis y saltar de una a otra sin pasar por las intermedias, como en el juego de los números de Tenenbaum o en DreamCoder;
- **reutilizar** una pieza en contextos distintos, sabiendo que es la misma;
- **componer** piezas sin que se mezclen.

Es lo que el .epub llamaba «combinatoria sin mezcla» y «búsqueda». Es también el ingrediente que diferencia a NLI de LPN. Que no exista un experimento que lo aísle (la fila vacía de la Figura 20) es, quizás, la mejor oportunidad de trabajo que deja este capítulo.

Qué desmentiría el principio

Un principio que no se puede desmentir no sirve. Este quedaría en problemas si se encontrara un caso donde lo discreto gane con capacidad, pérdida, esparsidad y margen igualados, y en una tarea que no requiera búsqueda ni composición. El experimento factorial de la sección anterior es justamente la forma de buscarlo.

Qué nos llevamos

- Cada pieza deja una firma distinta: la capacidad se borra al igualar bits, los modos solo aparecen con azar, el redondeo crece con el horizonte, la identidad exacta aparece al buscar o componer.
- Los confusores típicos son la pérdida, la capacidad, el tamaño del modelo, la arquitectura y el estimador del gradiente.
- Para P3: un diseño factorial (representación × capacidad × pérdida × entorno × horizonte, y geometría) separa las piezas. Aplicado a LPN y NLI, separa la discreción del largo variable.
- Para A1: casi todos los beneficios se reprodujeron sin discretizar. El principio candidato es **invarianza dentro, margen entre, cortes que coinciden con el mundo** (conjetura). Lo único sin sustituto continuo claro es la identidad exacta.

Capítulo VII

Colapso y error acumulado

En una frase *Hay varios colapsos, y en casi todos el gradiente encuentra una solución barata en la que el latente deja de llevar información; lo discreto no elimina el colapso, sino que lo transforma en uno que se puede contar y corregir, y el colapso y el error acumulado resultan ser los dos desenlaces de un mismo bucle.*

Este capítulo trata los costos. Responde a P1 (¿es más difícil el colapso con lo discreto?), a P5 (¿dónde ocurre el colapso del diccionario?) y a la nota del profesor (*«accumulated errors =?= collapse»*).

VII.1 Una familia de colapsos

La palabra «colapso» se usa en la literatura para cosas distintas. Conviene separarlas antes de preguntar si lo discreto las evita (Figura 23).

- **Colapso posterior** (semana 4): en un VAE, la posterior deja de depender de la entrada, $q(z|x) = p(z)$ para todo x, y el decodificador aprende a reconstruir sin mirar z.
- **Colapso completo**: el codificador entrega el mismo vector para toda entrada. Es el riesgo central de los métodos predictivos autosupervisados como BYOL o JEPA (capítulo II).
- **Colapso dimensional** (Jing et al., 2022): los vectores no son todos iguales, pero viven en un subespacio de pocas dimensiones. Se detecta porque muchos valores propios de su matriz de covarianza son casi cero. Es más sutil que el completo y aparece incluso en métodos contrastivos.
- **Colapso del diccionario**: de los K códigos, solo se usan unos pocos.
- **Colapso neuronal** (Papyan et al., 2020): al entrenar un clasificador mucho tiempo, las representaciones de cada clase se concentran en su promedio, y los promedios se ubican en los vértices de un símplex regular, todos a la misma distancia. Es el colapso «útil»: máxima invarianza dentro de cada clase y máximo margen entre clases.

Por qué el gradiente cae ahí

Los cuatro colapsos dañinos tienen un hilo común: existe una solución en la que el latente **no lleva información** y que, sin embargo, hace bajar la pérdida. El descenso de gradiente la encuentra por tres vías:

1. **Una solución trivial con pérdida baja.** Si el objetivo es predecir una representación que el propio modelo produce, una constante la predice perfecto. En un VAE, apagar el latente anula el término KL.

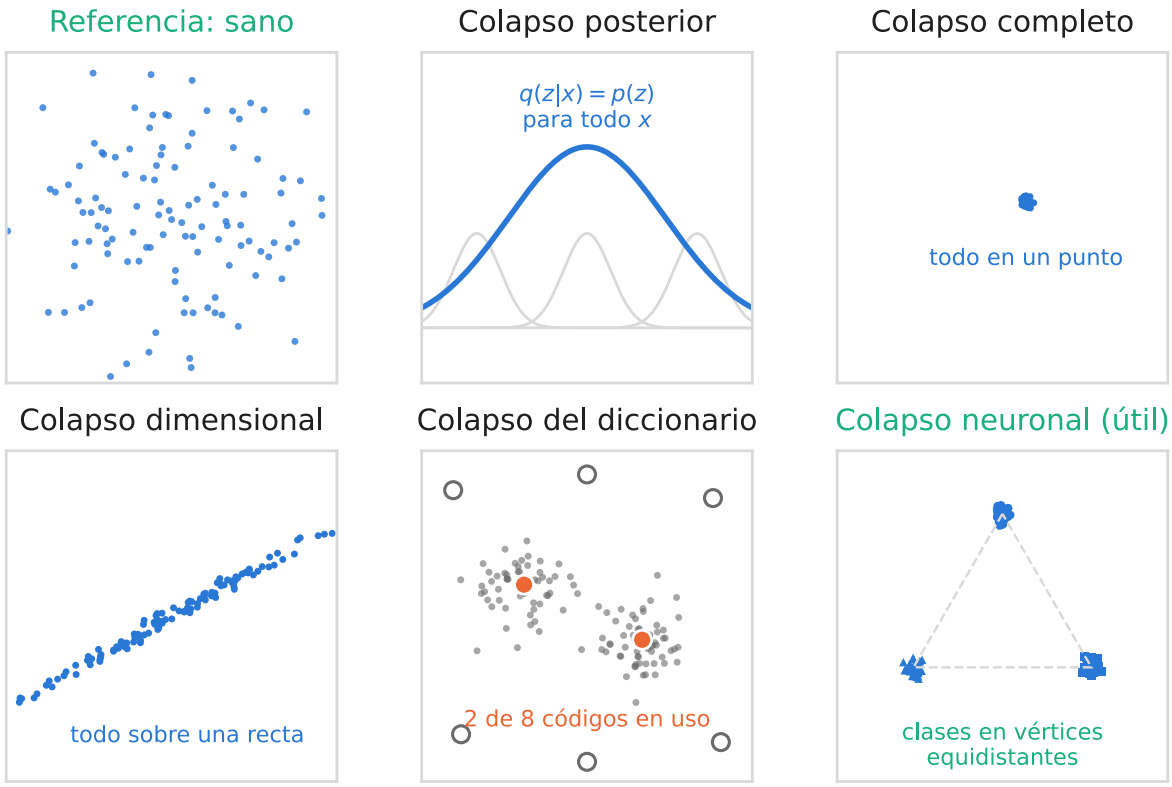


Figura 23. Cinco colapsos, más una referencia sana. En los cuatro del centro, el latente deja de distinguir entradas que debería distinguir. El colapso neuronal es distinto: deja de distinguir lo que **no** importa (las variaciones dentro de una clase) y conserva exactamente lo que importa.

2. **Una preferencia por soluciones simples.** El entrenamiento por gradiente tiende a usar pocas direcciones del espacio; Jing et al. muestran que eso, combinado con aumentos de datos fuertes, produce colapso dimensional.
3. **«El rico se hace más rico».** En VQ, cada punto solo actualiza el código más cercano. Un código que gana atrae más puntos, y uno que nunca gana nunca se mueve (lo vemos en detalle más abajo).

Los remedios continuos atacan estas vías. Los **negativos** del aprendizaje contrastivo castigan que dos imágenes distintas queden juntas. **EMA y stop-gradient** impiden que el modelo arrastre su propio objetivo hacia la constante. **VICReg** (Bardes et al., 2022) agrega términos explícitos: uno de **varianza**, que exige que cada coordenada varíe al menos γ entre ejemplos,

$$v(Z) = \frac{1}{d}\sum_{j=1}^{d} \max\left(0, \gamma - \sqrt{\operatorname{Var}(z_j) + \varepsilon}\right),$$

y uno de **covarianza**, que penaliza que dos coordenadas estén correlacionadas, para evitar el colapso dimensional. JEPA-Anything usa un «piso de actividad» por factor con la misma lógica.

VII.2 ¿Hace más difícil el colapso posterior lo discreto? (P1)

VQ-VAE: no colapsa, pero no por ser discreto

En un VAE, el término KL de la ELBO, $D_{\mathrm{KL}}(q(z|x) \parallel p(z))$, empuja la posterior hacia la prior. Si el decodificador es lo bastante potente para reconstruir sin ayuda, la solución más barata es igualarlas: eso es el colapso posterior.

En VQ-VAE, la posterior es **determinista** (toda la masa en el código más cercano) y la prior es **uniforme y fija** sobre K códigos. Entonces

$$D_{\mathrm{KL}}(q(z|x) \parallel p(z)) = \log K \quad \text{para todo } x,$$

una constante que no depende de los parámetros. **No hay ninguna fuerza que empuje hacia el colapso** ESTABLECIDO. Así lo plantean van den Oord et al. y lo anotamos en los apuntes de la semana 4.

Pero hay que mirar con cuidado qué produjo ese efecto. No fue la discreción, sino la combinación de **posterior determinista** y **prior fija**. La prueba es que un VAE **categórico** con posterior aprendida y estocástica (por ejemplo, con Gumbel-Softmax) sí puede colapsar: basta con que la posterior se vuelva uniforme, igual a la prior, para que el KL valga cero. DreamerV2 y V3 usan latentes categóricos y aun así necesitan defensas explícitas: el *KL balancing* y los *free bits*, que dejan de penalizar el KL por debajo de 1 nat para que el modelo no gane nada apagando el latente ESTABLECIDO.

«Partir con elementos bien separados»

La segunda parte de P1 pregunta si ayuda que los códigos estén bien separados desde el inicio. Los dos papers de la semana 4 permiten responder con precisión.

Identificabilidad. Wang et al. (2021) muestran que el colapso posterior ocurre **si y solo si** el latente no es **identificable**: si valores distintos de z producen la misma distribución de datos, los datos no pueden decir cuál z se usó, y el modelo puede ignorarlo. Con códigos bien separados **que el decodificador traduce a salidas distintas**, ver x revela z, y el colapso deja de ser una solución (Figura 24).

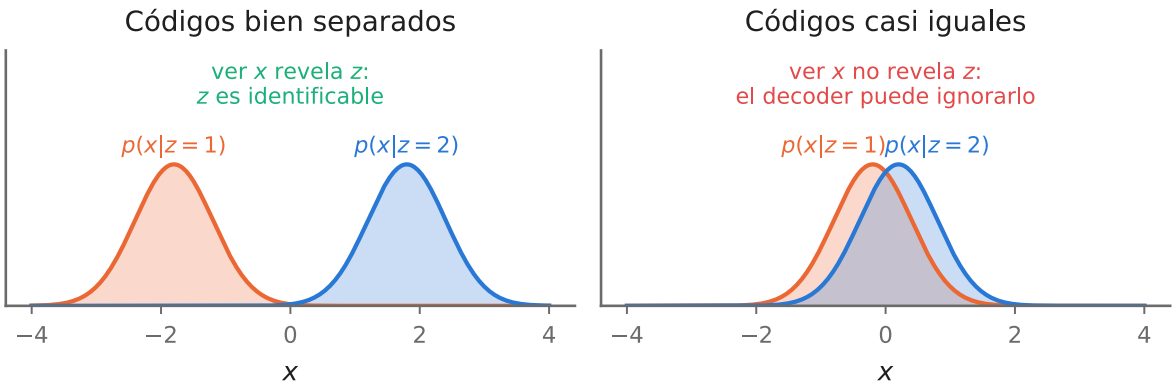


Figura 24. Identificabilidad. Si cada código produce datos claramente distintos (izquierda), los datos revelan el código y el decodificador no puede ignorarlo. Si los códigos producen datos casi iguales (derecha), el código no es identificable y el colapso posterior es una solución válida.

Transición de fase. Li et al. (2025) encuentran, para VAEs gaussianos, que el colapso ocurre cuando el ruido del decodificador supera la mayor varianza de los datos: $\sigma'^2 > \max_i \xi_i^2$, donde los ξ_i^2 son los valores propios de la covarianza de los datos. Cuando los datos forman grupos bien separados, la varianza **entre grupos** es grande, y el sistema queda lejos del umbral.

Respuesta a P1

Sí, con lo discreto el colapso posterior suele ser más difícil, pero no por la discreción en sí. En VQ-VAE desaparece la fuerza que empuja al colapso, porque el KL es constante; en un VAE categórico estocástico esa fuerza sigue ahí. Partir con códigos bien separados ayuda por **identificabilidad** y porque aleja el umbral de la transición de fase, con dos condiciones:

1. la separación tiene que **mantenerse** durante el entrenamiento: con un diccionario aprendido los códigos derivan y algunos mueren, mientras que con una grilla fija (FSQ) no;
2. la separación tiene que corresponder a **diferencias reales en los datos**: códigos separados que el decodificador traduce a lo mismo no sirven.

VII.3 Colapso del diccionario (P5)

El mecanismo

En VQ, cada salida del codificador actualiza **solo** al código más cercano. Si un código se inicializa lejos de los datos, nunca es el más cercano a nada, nunca recibe gradiente y nunca se mueve: es un **código muerto** (Figura 25, izquierda). Al revés, los códigos que ganan al principio atraen más puntos y se acomodan mejor, lo que los hace ganar más. Esta dinámica de «el rico se hace más rico» produce el colapso sin que nada lo empuje explícitamente.

FSQ ofrece la comparación más clara. Mentzer et al. (2024) reportan que el uso de VQ cae bajo el 50 % para diccionarios de más de 2^{11} códigos y que, en diccionarios grandes, no logra usar más de unos 2^{10}. FSQ, en cambio, usa casi todos sus códigos en 2^{14} y más de la mitad en 2^{16}, sin pérdidas

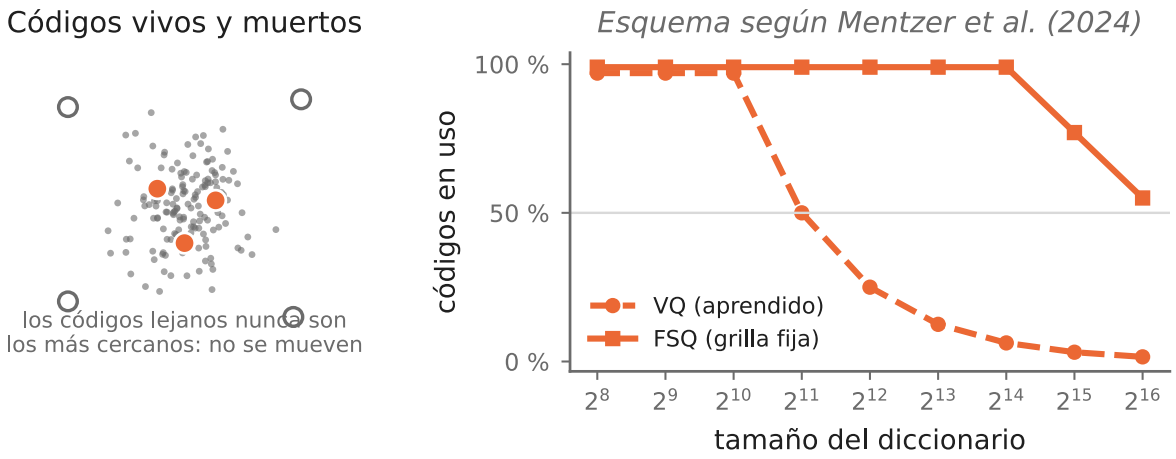


Figura 25. Izquierda: códigos vivos (naranjos) dentro de la nube de datos y códigos muertos (vacíos) fuera de ella. Derecha: esquema del uso del diccionario según su tamaño, a partir de lo que reportan Mentzer et al.: VQ no logra usar más de unos 2^{10} códigos, mientras que FSQ usa casi todos hasta 2^{14}.

auxiliares ni trucos ESTABLECIDO. La razón es estructural: en una grilla fija no hay códigos que puedan quedar lejos de los datos, porque nadie los mueve.

Dónde se ha documentado

Modelo	Dominio	Qué ocurre y cómo se combate
VQ-VAE (Oord et al., 2017)	imagen, audio	Propone actualizar los códigos con EMA (promedio de los puntos asignados) como alternativa a la pérdida del diccionario; el bajo uso se documentó en los trabajos siguientes.
Jukebox (Dhariwal et al., 2020)	música	Códigos poco usados; **reinicio aleatorio**: se reemplazan por salidas recientes del codificador.
VQGAN (Esser et al., 2021)	imagen	Solo el 2 % de 16.384 códigos en uso; el *rotation trick* lo sube al 27 % (Fifty et al., 2025).
ViT-VQGAN (Yu et al., 2022)	imagen	Bajo uso; se mejora buscando el código en un espacio de **baja dimensión** y con vectores **normalizados**.
SoundStream (Zeghidour et al., 2021)	audio	Códigos muertos en cuantización residual; inicialización con k--means y reemplazo de códigos sin uso.
wav2vec 2.0 (Baevski et al., 2020)	habla	Diccionarios aprendidos con Gumbel; **pérdida de diversidad** que premia usar los códigos por igual.
SwAV, DINO (Caron et al., 2020; Caron et al., 2021)	autosupervisado	Todas las imágenes asignadas al mismo prototipo; **reparto equitativo** (Sinkhorn) en SwAV, **centrado y afilado** en DINO.
Mezcla de expertos (Shazeer et al., 2017)	lenguaje	El enrutador elige siempre a los mismos pocos expertos; pérdida de **balance de carga**.

Tabla 4. Modelos donde se ha documentado el colapso del diccionario (o su equivalente).

Las dos últimas filas son las más interesantes. SwAV y DINO no son modelos «discretos»: discretizan **la salida** en una distribución sobre prototipos. La mezcla de expertos discretiza **la elección de una subred**. El colapso aparece en todas partes donde hay una elección entre K opciones y la elección ganadora recibe más señal. Por eso MoP-JEPA, que asigna cada ejemplo a una sola cabeza, enfrenta en principio el mismo riesgo.

Lo discreto transforma el colapso

Con esto se puede responder la versión general de P1. Lo discreto **no elimina** el colapso, **lo transforma**. Un colapso geométrico en continuo (puntos que se acercan de a poco, o un subespacio que se achica) se vuelve, en discreto, un colapso de **asignación**: muchas entradas caen en el mismo código. Este segundo tipo tiene dos ventajas prácticas:

- **se puede contar**: basta con medir la perplejidad o la fracción de códigos en uso;
- **tiene remedios directos**: reinicios, reparto equitativo o grillas fijas.

El colapso dimensional, en cambio, exige mirar el espectro de una matriz de covarianza para detectarlo. Es la misma idea del capítulo V aplicada al entrenamiento: **lo discreto hace visible el problema**.

VII.4 ¿Errores acumulados = colapso? (N)

Llegamos a la nota del profesor. En el capítulo V vimos cómo se acumulan los errores en un rollout; ahora tenemos los colapsos. ¿Son lo mismo?

Tres bucles

Hay tres situaciones en las que la salida de un modelo vuelve a entrar al mismo modelo, y en las tres aparecen **tanto** errores acumulados **como** colapsos:

1. **Rollouts.** La predicción es la entrada del paso siguiente. Con un predictor MSE, cada paso tira hacia el promedio condicional (capítulo II), y tras muchos pasos las predicciones se vuelven borrosas y parecidas entre sí. Es lo que MoP-JEPA llama «colapso» en las bifurcaciones.
2. **Bootstrapping.** En BYOL y JEPA, el objetivo lo produce (una copia lenta de) el propio codificador. Si el objetivo siguiera al codificador sin freno, el sistema podría deslizarse hacia la constante. EMA y *stop-gradient* frenan ese bucle.
3. **Entrenar con datos generados.** Shumailov et al. (2024) muestran que si cada generación de un modelo se entrena con textos o imágenes producidos por la anterior, los errores de estimación se acumulan generación tras generación. Las colas de la distribución se pierden, y el modelo termina produciendo una versión cada vez más estrecha de los datos: el «colapso de modelos».

La misma dinámica, dos desenlaces

Las tres situaciones son un **sistema dinámico**: aplicar una y otra vez una misma función. La Figura 26 muestra los tres comportamientos posibles.

- **Si el bucle contrae** (pendiente menor que 1), dos estados distintos se acercan en cada vuelta. Los errores desaparecen, **y con ellos la información**. Ese es el colapso.
- **Si el bucle expande**, dos estados cercanos se alejan. Las diferencias espurias crecen. Ese es el error acumulado, con su forma extrema, la divergencia.
- **Con redondeo**, el bucle contrae **dentro** de cada celda y deja intactas las diferencias **entre** celdas.

Respuesta a la nota del profesor

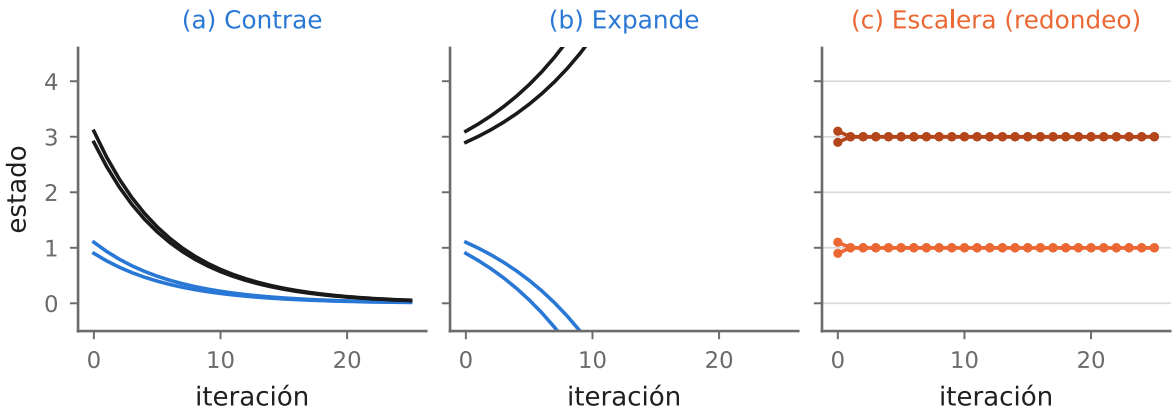


Figura 26. Iterar una función. (a) Si la función contrae, todos los estados convergen a un punto: los errores se apagan, pero también las diferencias reales (colapso). (b) Si expande, las diferencias crecen: los errores se amplifican (divergencia). (c) Con redondeo, cada celda contrae hacia su código, pero las celdas no se mezclan: los errores dentro de una celda se borran y las diferencias entre celdas se conservan.

No son lo mismo: el **error acumulado** es el crecimiento de diferencias que no deberían existir, y el **colapso** es la desaparición de diferencias que sí deberían existir. Pero son los **dos desenlaces de un mismo bucle** en el que la salida de un modelo vuelve a ser su entrada o su objetivo: el bucle expande (error acumulado) o contrae (colapso).

La discretización contrae solo dentro de cada celda, y por eso corrige errores sin colapsar. Sus dos modos de falla son las versiones discretas de los dos continuos:

- el **error silencioso** (un salto a otra celda) es el error acumulado del mundo discreto;
- el **colapso del diccionario** (todo cae en pocas celdas) es su colapso.

Para pensar *Con esta mirada, ¿qué hace exactamente la EMA del codificador objetivo en I-JEPA: contrae, expande o frena el bucle?*

Qué nos llevamos

- Hay cinco colapsos: posterior, completo, dimensional, del diccionario y neuronal. En los cuatro primeros, el latente deja de llevar información porque eso abarata la pérdida.
- P1: en VQ-VAE no hay fuerza hacia el colapso posterior porque el KL es constante; eso se debe a la posterior determinista y la prior fija, no a la discreción. Los códigos separados ayudan por identificabilidad, si la separación se mantiene y corresponde a diferencias reales.
- P5: el colapso del diccionario aparece en VQ-VAE, Jukebox, VQGAN, ViT-VQGAN, SoundStream, wav2vec 2.0, SwAV/DINO y en las mezclas de expertos; FSQ lo evita por construcción.
- Lo discreto transforma el colapso en uno contable y corregible.
- N: el error acumulado y el colapso son los dos desenlaces de un bucle de retroalimentación; el redondeo contrae dentro de las celdas y separa entre ellas.

Capítulo VIII

JEPA y discretización

En una frase *En la familia JEPA casi nadie discretiza los valores del estado: se discretiza la elección entre futuros, unos tokens semánticos, las acciones o la estructura, cada vez por una razón distinta, y todavía no existe una comparación limpia entre un JEPA discreto y uno continuo.*

Este capítulo responde a P4: ¿alguien discretiza en JEPA? ¿Por qué?

VIII.1 Recordatorio: qué hace un JEPA

I-JEPA (Assran et al., 2023) aprende representaciones de imágenes con una tarea simple (Figura 27, arriba):

1. Se toma una parte visible de la imagen (el contexto) y se codifica.
2. Un predictor intenta adivinar la **representación** de otras partes ocultas (los objetivos).
3. La representación de los objetivos la calcula un segundo codificador, actualizado por EMA, con *stop-gradient*.

La pérdida es la distancia cuadrática entre lo predicho y el objetivo, en el espacio de representaciones y no en píxeles. Sus ablaciones aportan dos datos útiles para nosotros. Predecir en espacio de representaciones en vez de píxeles sube la precisión en ImageNet con 1 % de etiquetas de 40,7 a 66,9 (tabla 7, aunque con 800 y 500 épocas de entrenamiento, respectivamente). Y un predictor **más angosto** rinde mejor que uno ancho, 70,7 frente a 68,4 (tabla 14) ESTABLECIDO. Ninguna de las dos mejoras discretiza nada: la primera es abstracción y la segunda es un cuello de botella continuo.

JEPA-Anything (Cui et al., 2026) agrega la factorización predictiva ortogonal (OPF). El objetivo se proyecta en K subespacios ortogonales (normalmente $K = 4$ o 5), cada factor tiene su propio predictor y el estado completo se recompone con una pseudoinversa. La ortogonalidad controla el número de condición de esa recomposición, y con ello **cuánto se amplifican** los errores de los factores. Es una vía para la estabilidad (pieza 5) que no discretiza valores, sino la estructura.

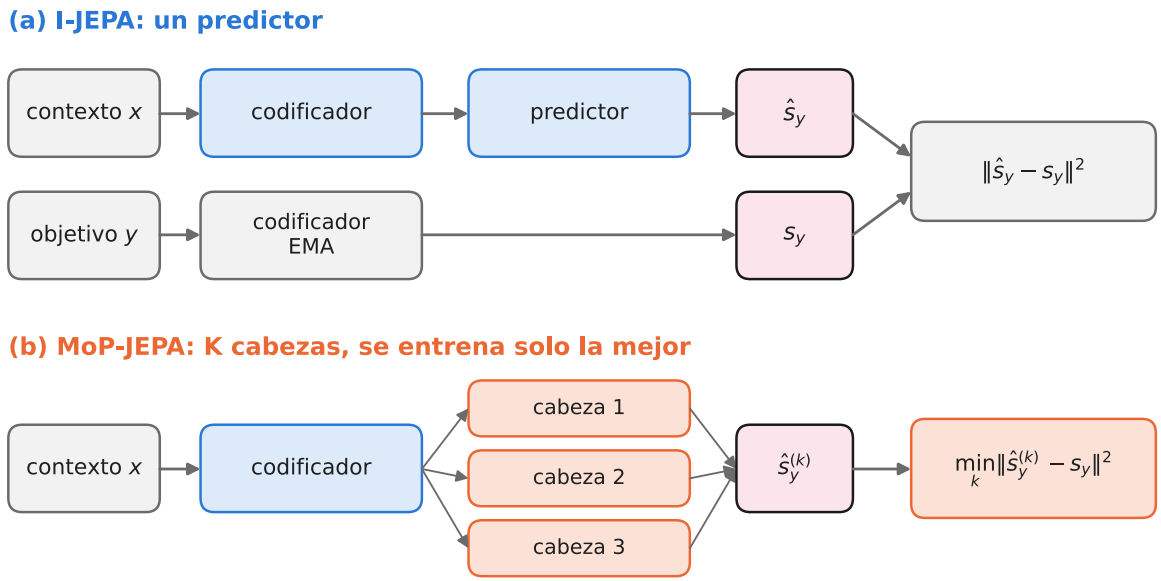


Figura 27. (a) I-JEPA: un predictor entrenado con distancia cuadrática, que en una bifurcación apunta al promedio. (b) MoP-JEPA: K cabezas, y cada ejemplo entrena solo a la que mejor lo predice.

VIII.2 Quién discretiza qué, y por qué

Trabajo	Qué discretiza	Por qué	Resultado
LeCun (2022)	la variable latente z del predictor (propuesta)	representar futuros múltiples y limitar cuánta información lleva z [4] [1]	propuesta conceptual, sin experimento
MoP-JEPA	qué cabeza predice	no promediar en bifurcaciones [4]	éxito de 0,85 frente a 0,02–0,09 en laberintos
Discrete-JEPA	tokens semánticos (VQ)	razonamiento simbólico; evitar error acumulado [5] [6]	peor al inicio y mejor a 1000 pasos; hiperparámetros distintos
DCWM	el latente (FSQ), sin reconstrucción	estocasticidad y clasificación [4] [3]	competitivo con TD-MPC2
Garrido et al.	acciones latentes (VQ, como baseline)	limitar la capacidad de la acción [1]	VQ falla; continuo con ruido o esparsidad funciona
JEPA-Anything	la estructura (K factores)	repartir la capacidad predictiva; estabilidad [5]	mejoras pequeñas y auditoría de estabilidad
DINO, SwAV	la salida: distribución sobre prototipos	evitar el colapso, con reparto equitativo o centrado	métodos de referencia en autosupervisado

Tabla 5. Discretización en la familia JEPA y en métodos cercanos.

LeCun (2022): la variable latente

En la propuesta original de LeCun (LeCun, 2022), un JEPA para modelos del mundo tiene, además del predictor, una **variable latente** z que representa lo que no se puede predecir desde el contexto,

por ejemplo cuál de varios futuros ocurrirá. Para que z no termine llevando **toda** la respuesta (que el predictor haga trampa copiando el objetivo en z), hay que limitar su información. Entre las formas que se discuten están hacerla discreta con pocos valores, de baja dimensión, esparsa o ruidosa. Con una z discreta, además, se pueden recorrer todos sus valores para planificar SUGERIDO. Son las piezas 4 y 1, y también las mismas opciones que Garrido et al. compararon años después para las acciones: allí ganaron las versiones continuas ruidosas o esparsas.

MoP-JEPA y Discrete-JEPA

MoP-JEPA (capítulo IV) discretiza **la elección**, con un predictor por modo. Es la implementación más directa de la z discreta de LeCun.

Discrete-JEPA (Baek et al., 2025) es el trabajo más cercano a un «JEPA discreto». Discretiza con VQ **solo** unas representaciones semánticas globales y mantiene continuas las de los parches. Lo motivan dos argumentos que ya conocemos: las tareas simbólicas requieren estructura discreta, y «las representaciones continuas sufren de errores acumulados en la predicción secuencial». Lo evalúan como modelo del mundo autorregresivo en secuencias sintéticas con patrones (colores que se repiten, pelotas que parpadean). El resultado tiene exactamente la firma del redondeo del capítulo V:

- en los primeros pasos, I-JEPA es **mejor**: en el paso 10, LPIPS de 0,0001 frente a 0,0028;
- a largo plazo, Discrete-JEPA es mucho mejor: en el paso 1000, LPIPS de 0,0242 frente a 0,1554;
- I-JEPA pierde la consistencia del patrón cerca del paso 600 SUGERIDO.

La etiqueta es SUGERIDO y no ESTABLECIDO porque los dos modelos usan dimensiones de token y tasas de aprendizaje distintas. Además, los autores reconocen que comparan casi solo contra I-JEPA.

DINO y SwAV: discretizar la salida para no colapsar

Los métodos de la familia *joint-embedding* más usados discretizan «en blando» su salida. Cada imagen se representa como una distribución (un softmax) sobre K prototipos aprendidos, y se pide que dos vistas de la imagen tengan distribuciones parecidas. Esto reaparece en el capítulo VII como colapso del diccionario: todas las imágenes podrían ir al mismo prototipo. SwAV (Caron et al., 2020) lo impide exigiendo que, en cada lote, las imágenes se repartan por igual entre prototipos; DINO (Caron et al., 2021), centrando y afilando las distribuciones. Aquí la estructura discreta no es la meta, sino la herramienta que hace **visible y controlable** el colapso.

VIII.3 Por qué discretizan, en resumen

Las razones se agrupan en cuatro, y cada una corresponde a una pieza:

- **Futuros múltiples** (pieza 4): LeCun, MoP-JEPA y DCWM.
- **Limitar información** (pieza 1): la z de LeCun y las acciones latentes. Aquí la evidencia favorece las vías continuas.
- **Horizontes largos y símbolos** (piezas 5 y 6): Discrete-JEPA.
- **Controlar el colapso**: DINO y SwAV.

Lo que **no** existe todavía es una comparación entre un JEPA con estado completamente discreto y uno continuo en igualdad de condiciones. El documento del profesor lo afirma, y tras revisar Discrete-JEPA lo mantenemos con un matiz: Discrete-JEPA se acerca, pero no iguala capacidad ni hiperparámetros. El experimento factorial del capítulo VI, aplicado a un JEPA, llenaría ese hueco.

Qué nos llevamos

- P4: sí se discretiza en JEPA, pero casi nunca el estado completo. Se discretizan la elección entre futuros (MoP-JEPA), tokens semánticos (Discrete-JEPA), el latente de un modelo sin reconstrucción (DCWM), las acciones (Garrido, donde lo discreto pierde), la estructura (JEPA-Anything) y la salida (DINO, SwAV).
- Las razones son cuatro: futuros múltiples, limitar información, horizontes largos o símbolos, y controlar el colapso.
- Las dos mejoras más claras de I-JEPA (predecir representaciones y estrechar el predictor) no discretizan nada.
- Discrete-JEPA muestra la firma del redondeo (peor al inicio, mejor a largo plazo), pero sin igualar condiciones sugerido.

Capítulo IX

¿Cuándo conviene discretizar?

En una frase *Conviene discretizar cuando la estructura relevante del problema es ella misma discreta (o se usa como si lo fuera) y cuando el sistema reutiliza sus propias salidas; no conviene cuando la información relevante vive en diferencias finas y continuas.*

Este capítulo responde a la pregunta abierta A2: ¿qué características comparten los espacios donde conviene discretizar, y aquellos donde no conviene? Como A1, no tiene una respuesta establecida. Lo que sigue es una síntesis de la evidencia en forma de criterios, cada uno con su pieza, su respaldo y, sobre todo, un **diagnóstico**: algo que se puede medir **antes** de decidir.

IX.1 Criterios a favor

El problema…	Evidencia	Pieza	Diagnóstico medible
requiere buscar, enumerar, reutilizar o componer hipótesis	DreamCoder, NLI frente a LPN	6	¿la tarea pide comparar candidatos exactamente o recombinar piezas?
tiene futuros que se ramifican	St. MuZero (2048 frente a Go), MoP-JEPA, DCWM	4	ajustar una mezcla a $p(z'\|z)$: ¿mejora la verosimilitud con más de un componente?
encadena muchas predicciones, con ruido por paso pequeño frente al margen	capítulo V, Discrete-JEPA, Meyer	5	estimar σ/d (error de un paso frente a la separación entre estados) y el horizonte; conviene si $\sigma/d \lesssim 0,2$
vive en un mundo con «huecos»: estados agrupados, categorías, saltos	Meyer (laberintos), DreamerV2 (hipótesis 4)	2 1	separabilidad de grupos en los estados: distancia entre grupos frente a la dispersión dentro de ellos
tiene poca capacidad frente a la complejidad del mundo	Meyer (escalamiento)	1 2	curva de error según el tamaño del modelo: ¿sigue bajando?
enfrenta un mundo que cambia	Meyer (aprendizaje continuo)	2	frecuencia y tamaño de los cambios del entorno

Tabla 6. Características compartidas por los casos en que discretizar ayudó.

IX.2 Criterios en contra

El problema…	Evidencia	Qué hacer en su lugar
necesita precisión fina en una variable continua y rica	MAR (texturas), Garrido (acciones reales)	continuo con capacidad limitada por ruido o esparsidad; difusión si hay varios modos
depende del orden o de interpolar	DCWM (*one-hot* y etiquetas fallan)	si se discretiza, entregar vectores del diccionario, no *one-hot*
optimiza por gradiente dentro del latente en tiempo de prueba	LPN	continuo, o discreto con relajación (Gumbel, como NLI)
tiene ruido por paso comparable al margen	capítulo V ($\sigma \gtrsim 0,29d$)	mejorar el modelo antes de redondear; o no redondear
necesita muchos bits por paso	colapso del diccionario en VQ grandes	FSQ o cuantización residual, si se insiste en discretizar

Tabla 7. Características compartidas por los casos en que discretizar perjudicó.

IX.3 Un árbol de decisión

La Figura 28 ordena los criterios como preguntas sucesivas. El orden no es arbitrario: va de lo que solo lo discreto ofrece (la identidad exacta) a lo que tiene alternativas continuas. Además, cada rama recomienda discretizar **lo mínimo necesario**: la elección y no el estado, el objetivo y no el latente, cuando eso basta.

IX.4 Un mapa

La Figura 29 ubica los casos del texto en dos ejes que resumen los criterios. El horizontal mide cuán «agrupado» es el mundo, desde suave y sin huecos hasta categórico y con saltos. El vertical mide cuánta precisión exige la tarea. Lo discreto gana a la derecha y abajo; lo continuo, a la izquierda y arriba. La ubicación es nuestro juicio y no una medición (conjetura).

Respuesta tentativa a A2

Los espacios donde conviene discretizar comparten una propiedad: su estructura relevante es **discreta o casi discreta** (categorías, ramas, objetos, reglas) y el sistema **reutiliza sus propias salidas** (rollouts largos, búsqueda, composición), de modo que el margen y la identidad exacta rinden una y otra vez. Los espacios donde no conviene comparten la propiedad opuesta: la información que importa vive en **diferencias finas y continuas** que ningún corte grueso respeta. En el medio, lo que mejor funciona es **híbrido**: un estado continuo con una elección discreta (Dreamer, MoP-JEPA) o con un objetivo discretizado (*two-hot*) (conjetura).

IX.5 Límites del marco

Tres advertencias:

- Los criterios son **correlaciones** entre la estructura del problema y el resultado en una docena de casos, no leyes. Varios casos tienen un solo paper detrás.

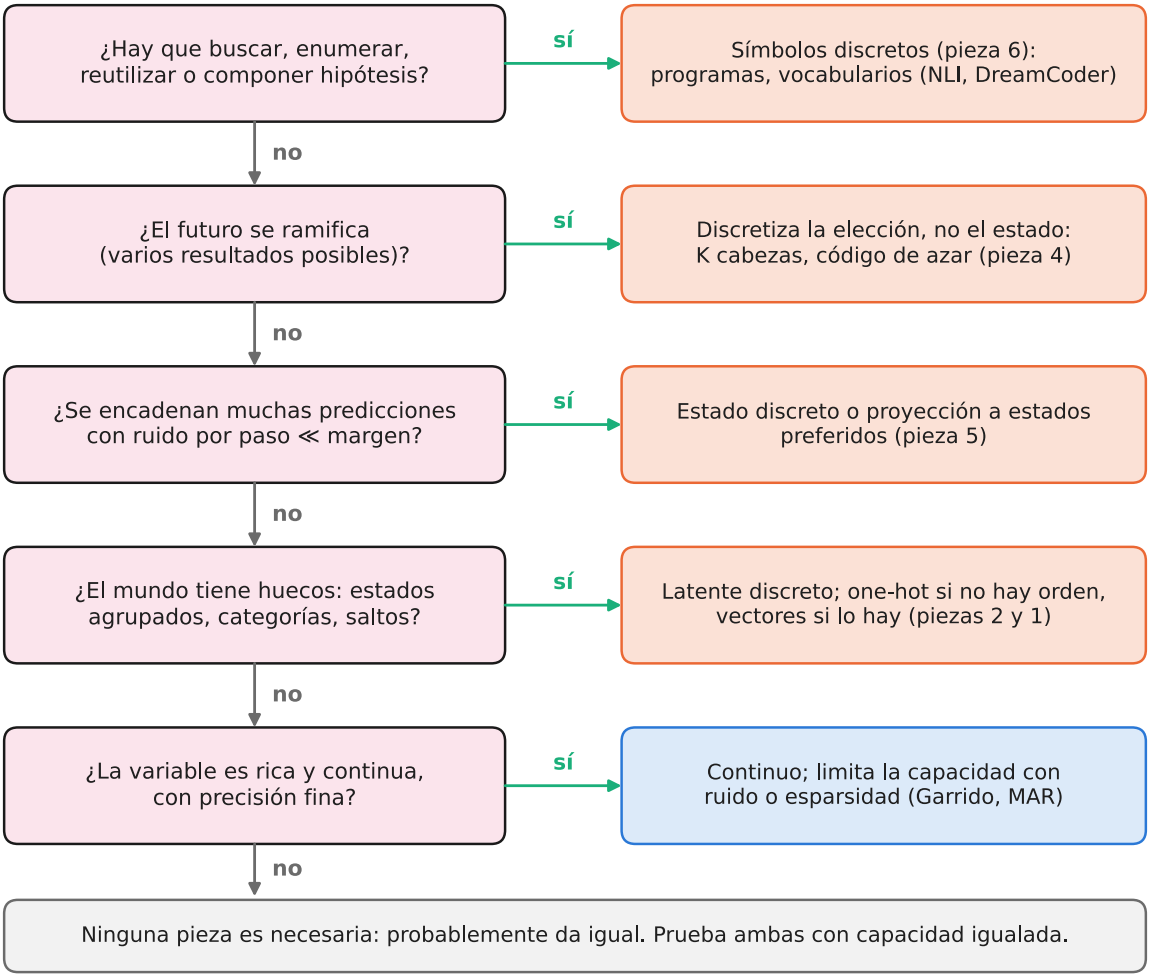


Figura 28. Árbol de decisión para discretizar. Cada «sí» recomienda qué discretizar (o qué no), y qué pieza se busca.

- Los diagnósticos suponen que ya se tiene alguna representación del problema para medir. Hay un círculo: para saber si el mundo tiene huecos, hay que haber elegido cómo mirarlo.
- El marco trata los cortes como fijos. Como dice el .epub, lo que distingue a la cognición humana quizás no es discretizar, sino **mover los cortes**: crear una categoría nueva cuando una vieja falla, o cambiar de escala. Ninguno de los modelos de este capítulo lo hace, salvo DreamCoder.

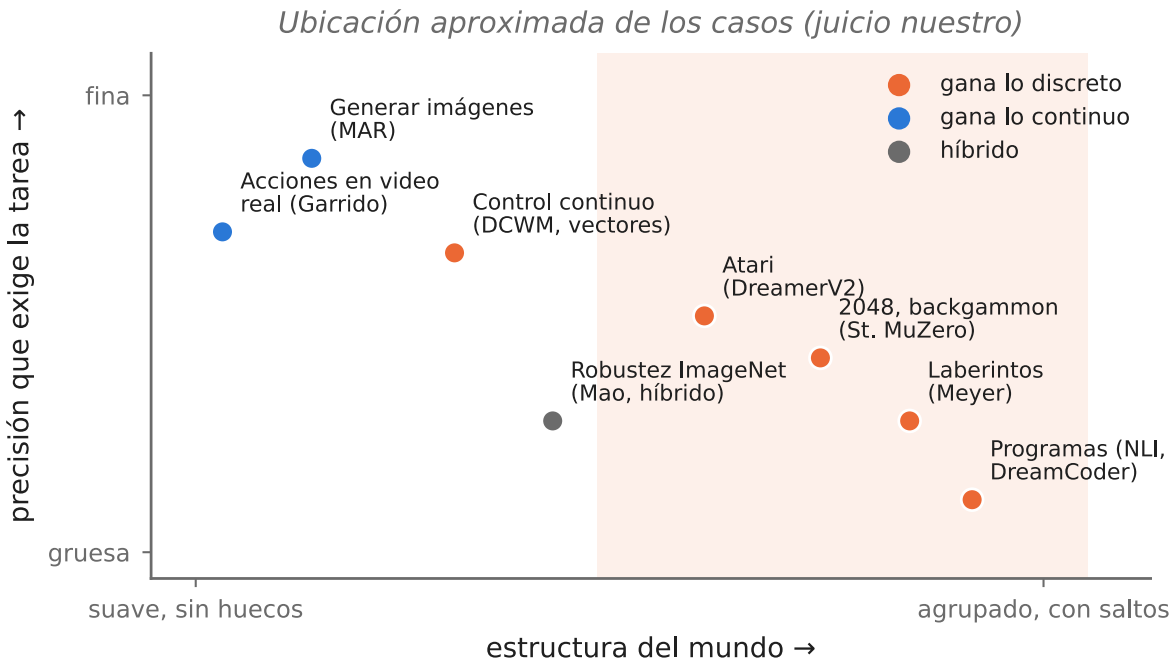


Figura 29. Los casos del capítulo en dos ejes. La franja naranja marca la región donde la evidencia favorece discretizar.

Qué nos llevamos

- A favor de discretizar: búsqueda o composición, futuros ramificados, cadenas largas con ruido pequeño frente al margen, mundos agrupados, poca capacidad y entornos cambiantes. Cada criterio tiene un diagnóstico medible.
- En contra: precisión fina en variables ricas, orden o interpolación (usar vectores, no *one-hot*), optimización por gradiente en el latente y ruido por paso comparable al margen.
- Regla práctica: discretizar **lo mínimo** que entregue la pieza necesaria; los diseños híbridos son los que tienen más respaldo.

Capítulo X

Conclusiones

En una frase *Discretizar no es una causa sino un paquete; casi todo lo que ofrece se puede conseguir por otras vías, salvo la identidad exacta, y la ventaja aparece cuando los cortes coinciden con el mundo y el sistema reutiliza sus propias salidas.*

X.1 Respuestas cortas

P1. *¿Es más difícil el colapso con lo discreto? ¿Ayuda partir con códigos separados?*
Más difícil, sí, pero no por la discreción. En VQ-VAE el KL es constante y no empuja al colapso posterior, gracias a la posterior determinista y la prior fija; un VAE categórico estocástico sí puede colapsar, y Dreamer necesita *free bits*. Los códigos separados ayudan por identificabilidad, si la separación se mantiene (grilla fija) y corresponde a diferencias reales en los datos. En general, lo discreto transforma el colapso en uno de asignación, que se puede contar y corregir. (Capítulo VII.) ESTABLECIDO para el KL constante; SUGERIDO para el resto.

P2. *¿Qué evidencia hizo creer que discretizar ayuda? ¿Dónde gana lo continuo?*
Nació de DreamerV2 (mejor en 42 de 55 juegos con latentes categóricos), cuya ablación no separa causas. Los experimentos más limpios muestran ventajas condicionales: con poca capacidad y en mundos cambiantes (Meyer), con azar (Stochastic MuZero, MoP-JEPA) y con la geometría adecuada (DCWM). Lo continuo gana en generación de imágenes (MAR) y en acciones reales (Garrido). (Capítulo IV.) ESTABLECIDO

P3. *¿Qué experimento comprueba o desmiente que la causa es la discretización?*
Uno que cambie una pieza a la vez. Proponemos un diseño factorial que cruza representación, capacidad igualada o libre, pérdida, entorno con o sin azar y horizonte. Aplicado a LPN y NLI, separa la discreción del largo variable. Cada pieza predice un patrón distinto de resultados. (Capítulo VI.)

P4. *¿Alguien discretiza en JEPA? ¿Por qué?*
Sí, pero casi nunca el estado completo. Se discretizan la elección entre futuros (MoP-JEPA), tokens semánticos (Discrete-JEPA), el latente sin reconstrucción (DCWM), las acciones (Garrido, donde pierde lo discreto), la estructura (JEPA-Anything) y la salida (DINO, SwAV). Las razones son futuros múltiples, limitar información, horizontes largos o símbolos, y controlar el colapso. (Capítulo VIII.)

P5. *¿Dónde ocurre el colapso del diccionario?*
Donde haya una elección entre K opciones en la que el ganador recibe más señal: VQ-VAE, Jukebox, VQGAN (2 % de uso), ViT-VQGAN, SoundStream, wav2vec 2.0, SwAV y DINO, y las mezclas de expertos. FSQ lo evita por construcción. (Capítulo VII.) ESTABLECIDO

N. *¿Errores acumulados = colapso?*
No son lo mismo, pero son los dos desenlaces de un mismo bucle de retroalimentación. Si el bucle expande, el error se acumula; si contrae, el sistema colapsa. El redondeo contrae dentro de cada celda y separa entre ellas, y sus fallas (error silencioso y colapso del diccionario) son las versiones discretas de las dos continuas. (Capítulos V y VII.) CONJETURA

E. *¿Qué significa que la discretización permite seguir el error?*
Que los estados válidos son pocos y separados, y entonces el error se puede **detectar** (caer entre estados), **corregir** (volver al más cercano), **localizar** (el primer paso que no coincide exactamente) y **contar**. Solo conviene si el ruido por paso es claramente menor que la mitad de la separación; desde $\sigma \approx 0,29d$, redondear empeora las cosas. (Capítulo V.)

A1. *¿Es la discreción o un principio más allá?*
Casi todos los beneficios se reprodujeron sin discretizar. El principio candidato es **invarianza dentro de cada clase, margen entre clases y cortes que coinciden con el mundo**; la discretización es su forma extrema. Lo único sin sustituto continuo claro es la identidad exacta: enumerar, reutilizar y componer. (Capítulo VI.) CONJETURA

A2. *¿Qué comparten los espacios donde conviene discretizar?*
Estructura relevante discreta o casi discreta, y un sistema que reutiliza sus propias salidas. No conviene cuando lo que importa vive en diferencias finas y continuas. En el medio, los diseños híbridos son los que tienen más respaldo. (Capítulo IX.) CONJETURA

X.2 Qué significa para nuestro proyecto

- **LPN y NLI.** El salto de LPN a NLI cambia al menos tres cosas: la discreción, el largo variable y el intérprete secuencial. Según este capítulo, la pieza que mejor explicaría una ventaja en **composición** es la identidad exacta [6], justo la que ningún experimento publicado aísla. Las variantes

del capítulo VI (LPN con largo variable, NLI con tokens continuos de capacidad igualada) son un experimento factible y nuevo.

- **DreamerV3.** Su diseño (memoria continua, elección categórica, objetivos *two-hot*, free bits) es el híbrido con más respaldo. Combina las piezas 4 y 3 sin discretizar lo que necesita precisión.
- **JEPA-Anything.** La ortogonalidad de sus factores es una vía continua para la estabilidad (5) y el margen entre factores. Pero fija K y no se pregunta cuándo crear un factor nuevo: es una discretización de la estructura con cortes que no se mueven.
- **La segunda pregunta del .epub.** El .epub cerraba con la idea de que la ventaja humana quizás no está en discretizar, sino en **saber cuándo volver a discretizar**. Este capítulo le da una forma más concreta. Si el beneficio viene de que los cortes coincidan con el mundo, entonces mover los cortes cuando dejan de coincidir es la habilidad central. Esa es la pregunta sobre refinar o ampliar.

X.3 Preguntas para la reunión

1. ¿Vale la pena montar el experimento factorial sobre LPN y NLI, o una versión reducida (solo NLI con tokens continuos de capacidad igualada)?
2. El colapso neuronal es una discretización que **emerge** del gradiente. ¿Es un buen punto de entrada para la segunda pregunta (de dónde salen las reglas)?
3. ¿Cómo se podría aislar la identidad exacta (pieza 6) sin cambiar a la vez la capacidad ni la arquitectura?
4. ¿Discrete-JEPA, o su repetición con condiciones igualadas, como próxima lectura o experimento?

Glosario

Ablación experimento que quita o cambia una sola pieza de un método para medir su aporte.
Atractor estado estable de una dinámica, al que convergen los estados cercanos (redes de Hopfield).
Bootstrapping usar las salidas del propio modelo como objetivo de entrenamiento (BYOL, JEPA).
Celda de Voronoi región de puntos más cercanos a un código que a cualquier otro.
Código muerto código de un diccionario que nunca es elegido y, por lo tanto, nunca se actualiza.
Colapso (del diccionario, posterior, completo, dimensional, neuronal) ver el capítulo VII.
Confusor variable que cambia junto con la que se estudia y podría explicar el efecto por sí sola.
EMA promedio móvil exponencial; en JEPA, forma de actualizar lentamente el codificador objetivo.
Entropía cruzada (CE) pérdida de clasificación; su óptimo es la distribución completa de resultados.
Error compuesto error que crece al usar cada predicción como entrada de la siguiente.
Error silencioso error tan grande que el redondeo lleva el estado a otro código válido sin detectarlo.
FSQ cuantización que redondea cada coordenada a una grilla fija de niveles.
Identificabilidad propiedad de un modelo en que valores distintos del latente producen datos distinguibles.
Margen distancia entre representaciones que deben distinguirse, comparada con el ruido.
MSE error cuadrático medio; su óptimo es el promedio de los resultados posibles.
One-hot vector con un 1 en la posición de la categoría y 0 en el resto.
Perplejidad 2^H; número efectivo de códigos usados.
Pieza cada uno de los seis cambios que trae la discretización (capítulo I).
Rollout secuencia de estados imaginados por un modelo del mundo.
STE (*straight-through estimator*) redondear hacia adelante y copiar el gradiente hacia atrás.
Two-hot representación de un número real como peso repartido entre dos intervalos vecinos.
VQ cuantización vectorial; reemplazar un vector por el más cercano de un diccionario aprendido.

Referencias

Antonoglou, I., Schrittwieser, J., Ozair, S., Hubert, T., & Silver, D. (2022). Planning in Stochastic Environments with a Learned Model. *International Conference on Learning Representations*. https://openreview.net/forum?id=X6D9bAHhBQ1

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. https://arxiv.org/abs/2301.08243

Baek, J., Lee, H., Hoang, C., Ren, M., & Ahn, S. (2025). Discrete JEPA: Learning Discrete Token Representations without Reconstruction. *ICML 2025 Workshop on Tokenization*. https://arxiv.org/abs/2506.14373

Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2006.11477

Bardes, A., Ponce, J., & LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *International Conference on Learning Representations*. https://arxiv.org/abs/2105.04906

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv:1308.3432*. https://arxiv.org/abs/1308.3432

Bruce, J., Dennis, M., Edwards, A., Parker-Holder, J., Shi, Y., Hughes, E., Lai, M., Mavalankar, A., Steigerwald, R., Apps, C., Aytar, Y., Bechtle, S., Behbahani, F., Chan, S., Heess, N., Gonzalez, L., Osindero, S., Ozair, S., Reed, S., … Rocktäschel, T. (2024). Genie: Generative Interactive Environments. *International Conference on Machine Learning*. https://arxiv.org/abs/2402.15391

Caron, M., Misra, I., Mairal, J., Goyal, P., Bojanowski, P., & Joulin, A. (2020). Unsupervised Learning of Visual Features by Contrasting Cluster Assignments. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2006.09882

Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging Properties in Self-Supervised Vision Transformers. *IEEE/CVF International Conference on Computer Vision*. https://arxiv.org/abs/2104.14294

Cui, T., Wang, Z., Xu, X., Liu, W., Yu, Z., Zhang, Y., Gao, Q., Yang, M., Ouyang, W., Heng, P. A., Wu, Y., Yin, Z., & Yang, L. (2026). JEPA-Anything: Learning Predictive Models across Different Worlds. *arXiv:2609.20800*. https://arxiv.org/abs/2609.20800

Dhariwal, P., Jun, H., Payne, C., Kim, J. W., Radford, A., & Sutskever, I. (2020). Jukebox: A Generative Model for Music. *arXiv:2005.00341*. https://arxiv.org/abs/2005.00341

Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., & Tenenbaum, J. B. (2021). DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. *ACM SIGPLAN Conference on Programming Language Design and Implementation*. https://arxiv.org/abs/2006.08381

Esser, P., Rombach, R., & Ommer, B. (2021). Taming Transformers for High-Resolution Image Synthesis. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. https://arxiv.org/abs/2012.09841

Farebrother, J., Orbay, J., Vuong, Q., Ali Taïga, A., Chebotar, Y., Xiao, T., Irpan, A., Levine, S., Castro, P. S., Faust, A., Kumar, A., & Agarwal, R. (2024). Stop Regressing: Training Value Functions via Classification for Scalable Deep RL. *International Conference on Machine Learning*. https://arxiv.org/abs/2403.03950

Fifty, C., Junkins, R. G., Duan, D., Iyengar, A., Liu, J. W., Amid, E., Thrun, S., & Ré, C. (2025). Restructuring Vector Quantization with the Rotation Trick. *International Conference on Learning Representations*. https://arxiv.org/abs/2410.06424

Garrido, Q., Nagarajan, T., Terver, B., Ballas, N., LeCun, Y., & Rabbat, M. (2026). Learning Latent Action World Models In The Wild. *International Conference on Machine Learning*. https://arxiv.org/abs/2601.05230

Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z. D., Gheshlaghi Azar, M., Piot, B., Kavukcuoglu, K., Munos, R., & Valko, M. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2006.07733

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning Latent Dynamics for Planning from Pixels. *International Conference on Machine Learning*. https://arxiv.org/abs/1811.04551

Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2021). Mastering Atari with Discrete World Models. *International Conference on Learning Representations*. https://arxiv.org/abs/2010.02193

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering Diverse Control Tasks through World Models. *Nature*, *640*, 647-653. https://arxiv.org/abs/2301.04104

Hamming, R. W. (1950). Error Detecting and Error Correcting Codes. *Bell System Technical Journal*, *29*(2), 147-160.

Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, Robust World Models for Continuous Control. *International Conference on Learning Representations*. https://arxiv.org/abs/2310.16828

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/1706.08500

Hopfield, J. J. (1982). Neural Networks and Physical Systems with Emergent Collective Computational Abilities. *Proceedings of the National Academy of Sciences*, *79*(8), 2554-2558.

Imani, E., & White, M. (2018). Improving Regression Performance with Distributional Losses. *International Conference on Machine Learning*. https://arxiv.org/abs/1806.04613

Jang, E., Gu, S., & Poole, B. (2017). Categorical Reparameterization with Gumbel-Softmax. *International Conference on Learning Representations*. https://arxiv.org/abs/1611.01144

Jing, L., Vincent, P., LeCun, Y., & Tian, Y. (2022). Understanding Dimensional Collapse in Contrastive Self-supervised Learning. *International Conference on Learning Representations*. https://arxiv.org/abs/2110.09348

Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, *82*(1), 35-45.

Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *International Conference on Learning Representations*. https://arxiv.org/abs/1312.6114

LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. *OpenReview*. https://openreview.net/forum?id=BZ5a1r-kVsf

Li, T., Tian, Y., Li, H., Deng, M., & He, K. (2024). Autoregressive Image Generation without Vector Quantization. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2406.11838

Li, Z., Zhang, F., Zhang, Z., & Chen, Y. (2025). Posterior Collapse as a Phase Transition in Variational Autoencoders. *arXiv:2510.01621*. https://arxiv.org/abs/2510.01621

Macfarlane, M. V., & Bonnet, C. (2025). Searching Latent Program Spaces. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2411.08706

Macfarlane, M. V., Bonnet, C., Hoof, H. van, & Lelis, L. H. S. (2026). Gradient-Based Program Synthesis with Neurally Interpreted Languages. *International Conference on Learning Representations*.

Mao, C., Jiang, L., Dehghani, M., Vondrick, C., Sukthankar, R., & Essa, I. (2022). Discrete Representations Strengthen Vision Transformer Robustness. *International Conference on Learning Representations*. https://arxiv.org/abs/2111.10493

Mentzer, F., Minnen, D., Agustsson, E., & Tschannen, M. (2024). Finite Scalar Quantization: VQ-VAE Made Simple. *International Conference on Learning Representations*. https://arxiv.org/abs/2309.15505

Meyer, E., White, A., & Machado, M. C. (2024). Harnessing Discrete Representations for Continual Reinforcement Learning. *Reinforcement Learning Journal*, *2*, 606-628. https://arxiv.org/abs/2312.01203

Oord, A. van den, Vinyals, O., & Kavukcuoglu, K. (2017). Neural Discrete Representation Learning. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/1711.00937

Pan, Y., Banman, K., & White, M. (2021). Fuzzy Tiling Activations: A Simple Approach to Learning Sparse Representations Online. *International Conference on Learning Representations*. https://arxiv.org/abs/1911.08068

Papyan, V., Han, X. Y., & Donoho, D. L. (2020). Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training. *Proceedings of the National Academy of Sciences*, *117*(40), 24652-24663. https://arxiv.org/abs/2008.08186

Petrache, M. (2026). Discrete vs. Continuous Representations in World Models: Pros and Cons. *Documento interno generado con IA, compartido el 20 de octubre de 2026.*

Scannell, A., Nakhaeinezhadfard, M., Kujanpää, K., Zhao, Y., Luck, K. S., Solin, A., & Pajarinen, J. (2025). Discrete Codebook World Models for Continuous Control. *International Conference on Learning Representations*. https://arxiv.org/abs/2503.00653

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., & Silver, D. (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, *588*, 604-609. https://arxiv.org/abs/1911.08265

Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, *27*(3), 379-423.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *International Conference on Learning Representations*. https://arxiv.org/abs/1701.06538

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., & Gal, Y. (2024). AI Models Collapse when Trained on Recursively Generated Data. *Nature*, *631*, 755-759.

Song, Z., Xing, X., Tang, Z., Huang, H., Huang, J., Yan, W., Lv, T., Yang, M., Niu, Z., He, B., Wang, L., & Yao, J. (2026). MoP-JEPA: Hard-Assigned Predictor Mixtures for Stochastic JEPA World Models. *arXiv:2607.05238*. https://arxiv.org/abs/2607.05238

Wang, Y., Blei, D. M., & Cunningham, J. P. (2021). Posterior Collapse and Latent Variable Non-identifiability. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2301.00537

Yu, J., Li, X., Koh, J. Y., Zhang, H., Pang, R., Qin, J., Ku, A., Xu, Y., Baldridge, J., & Wu, Y. (2022). Vector-quantized Image Modeling with Improved VQGAN. *International Conference on Learning Representations*. https://arxiv.org/abs/2110.04627

Yu, L., Lezama, J., Gundavarapu, N. B., Versari, L., Sohn, K., Minnen, D., Cheng, Y., Birodkar, V., Gupta, A., Gu, X., Hauptmann, A. G., Gong, B., Yang, M.-H., Essa, I., Ross, D. A., & Jiang, L. (2024). Language Model Beats Diffusion: Tokenizer is Key to Visual Generation. *International Conference on Learning Representations*. https://arxiv.org/abs/2310.05737

Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., & Tagliasacchi, M. (2021). SoundStream: An End-to-End Neural Audio Codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*. https://arxiv.org/abs/2107.03312